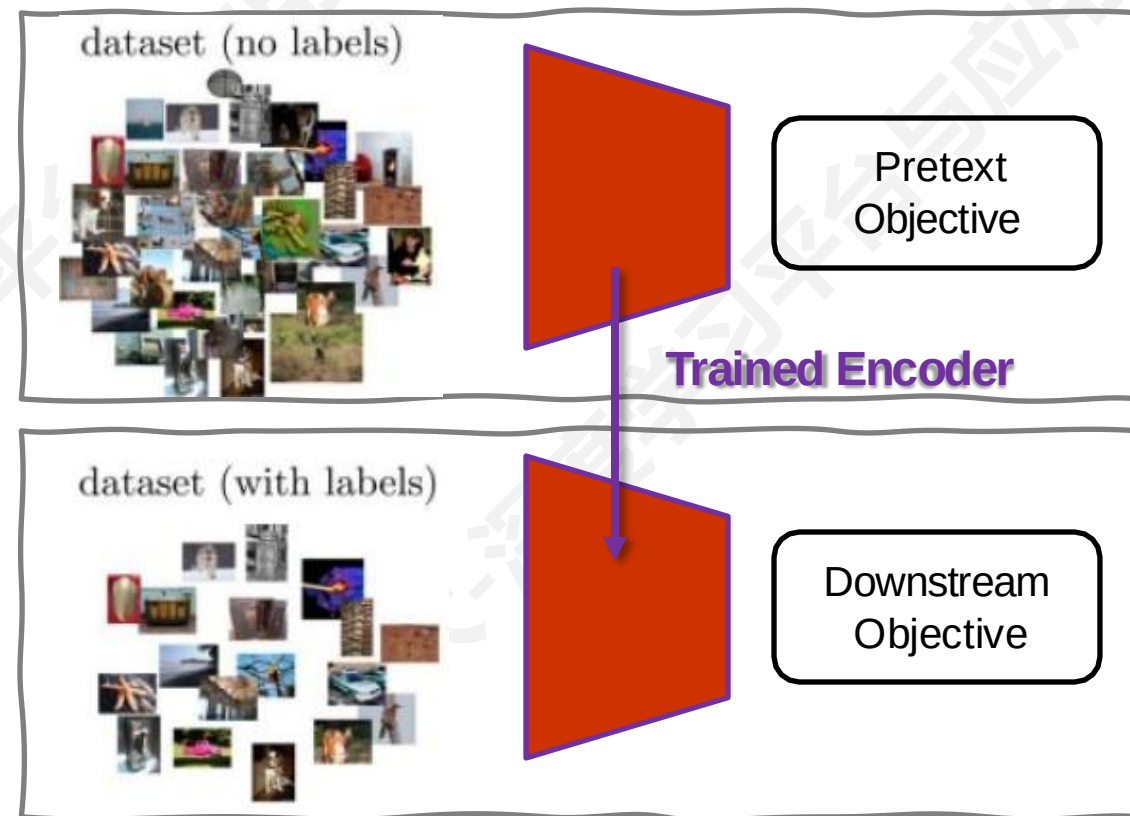




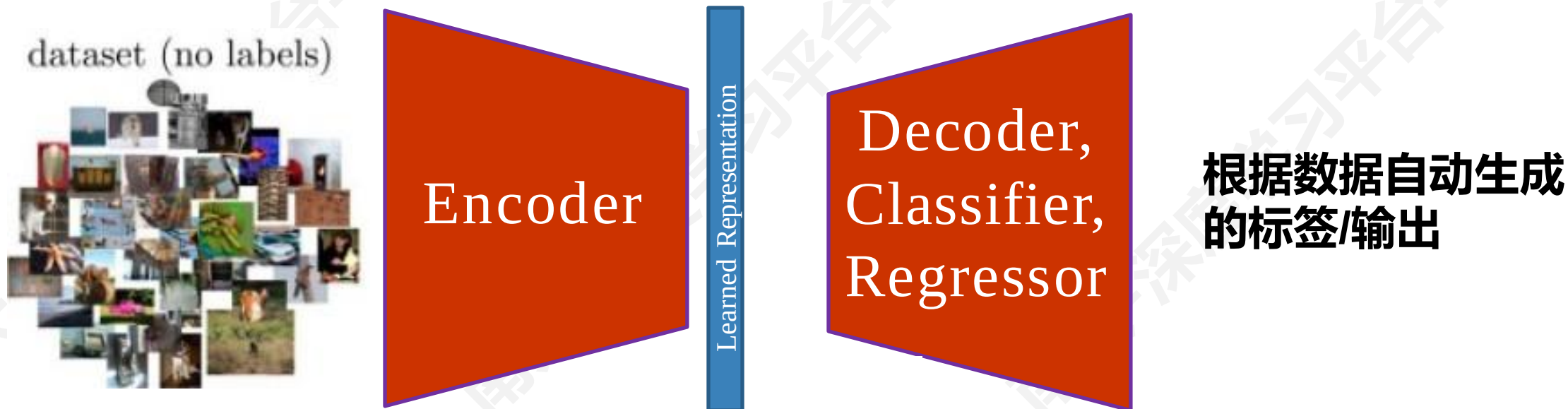
深度学习平台与应用

第十三讲：自监督学习

- **Pretext Task**
 - 根据数据本身定义任务
 - 无需注释的无监督任务
- **Downstream Task**
 - 想要应用的任务
 - 没有大规模数据集
 - 数据集有标注



■ Pretext Task



■ Downstream Task

dataset (with labels)



Encoder

Learned Representation

FC

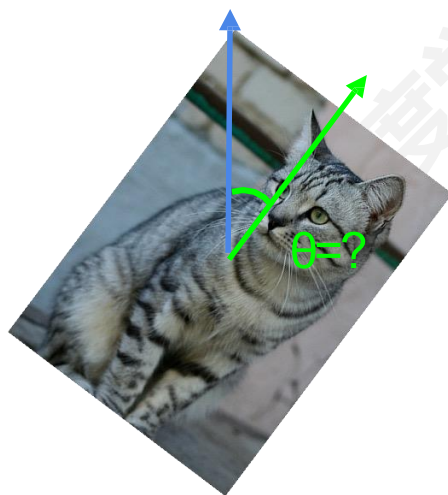
人工标注的标签/输出

■ Pretext Task

- 可以让模型学习到好的特征
- 可以自动为 Pretext Task 生成标签



image completion



rotation prediction



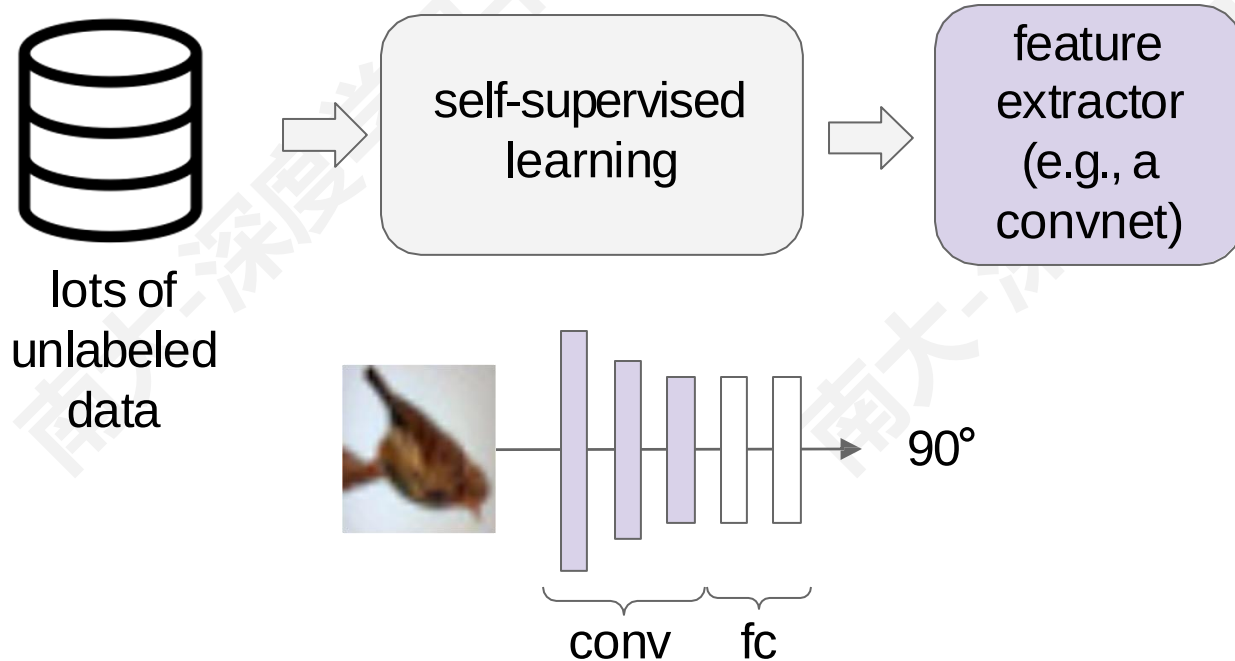
“jigsaw puzzle”



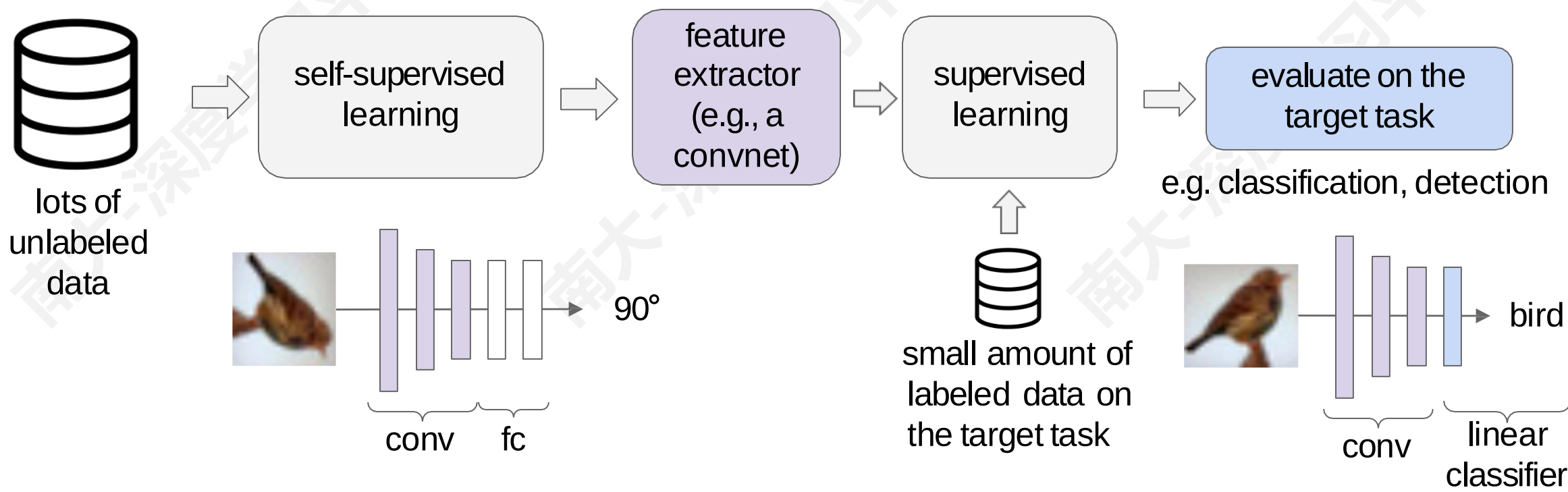
colorization

- 如何评估自监督学习方法?
 - Pretext Task 性能
 - 特征质量：线性分类效果、聚类、t-SNE 可视化
 - 鲁棒性和泛化性：不同数据集和不同变化
 - 计算效率：训练时间和训练所需资源
 - 迁移学习和 Downstream Task 性能

- 如何评估自监督学习方法？
 - 从 Pretext Task 中学习好的特征提取器



- 如何评估自监督学习方法？
 - 从 Pretext Task 中学习好的特征提取器
 - 使用少量有标签数据在目标任务上训练浅层网络

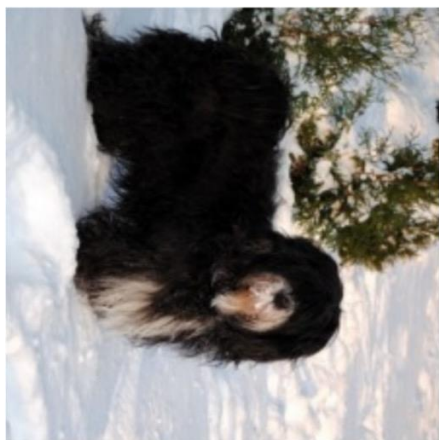


■ 预测旋转角度

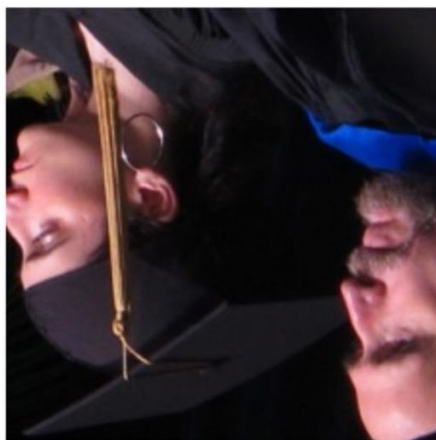
- 假设：只有当模型对正常状态的物体有“视觉常识”时，它才能识别出物体的正确旋转



90° rotation



270° rotation



180° rotation

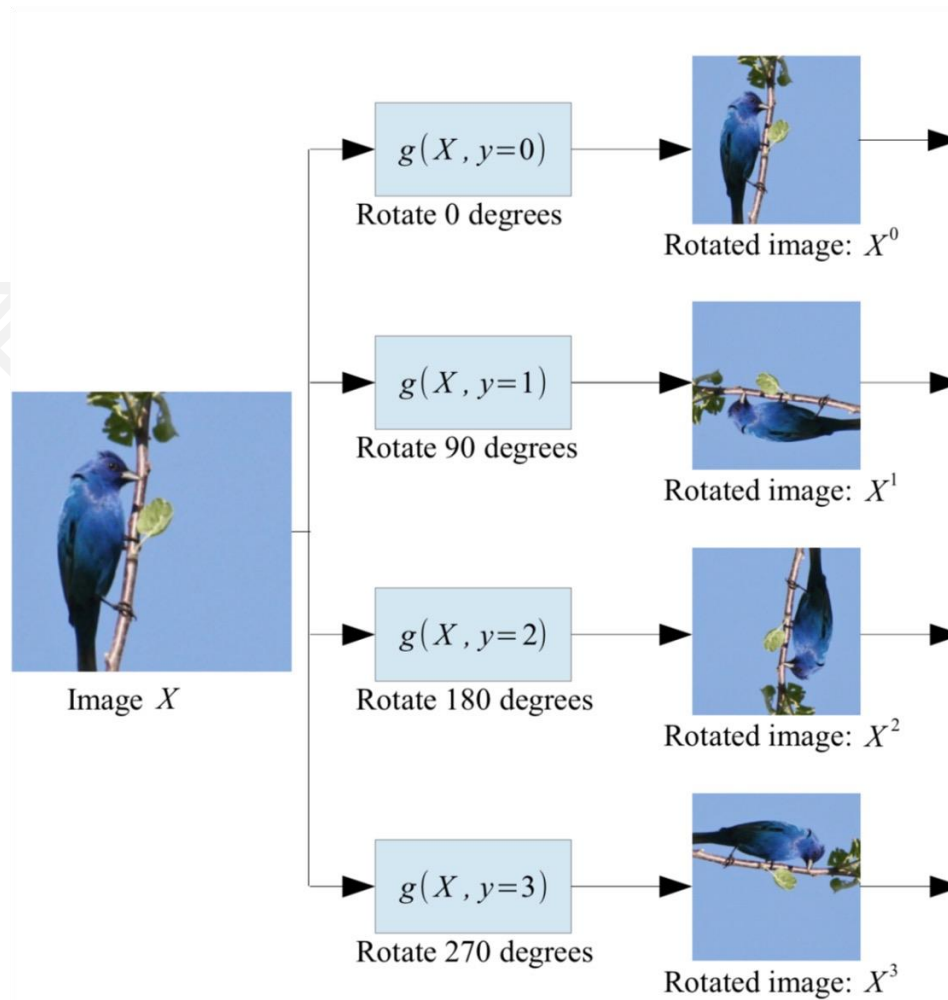


0° rotation

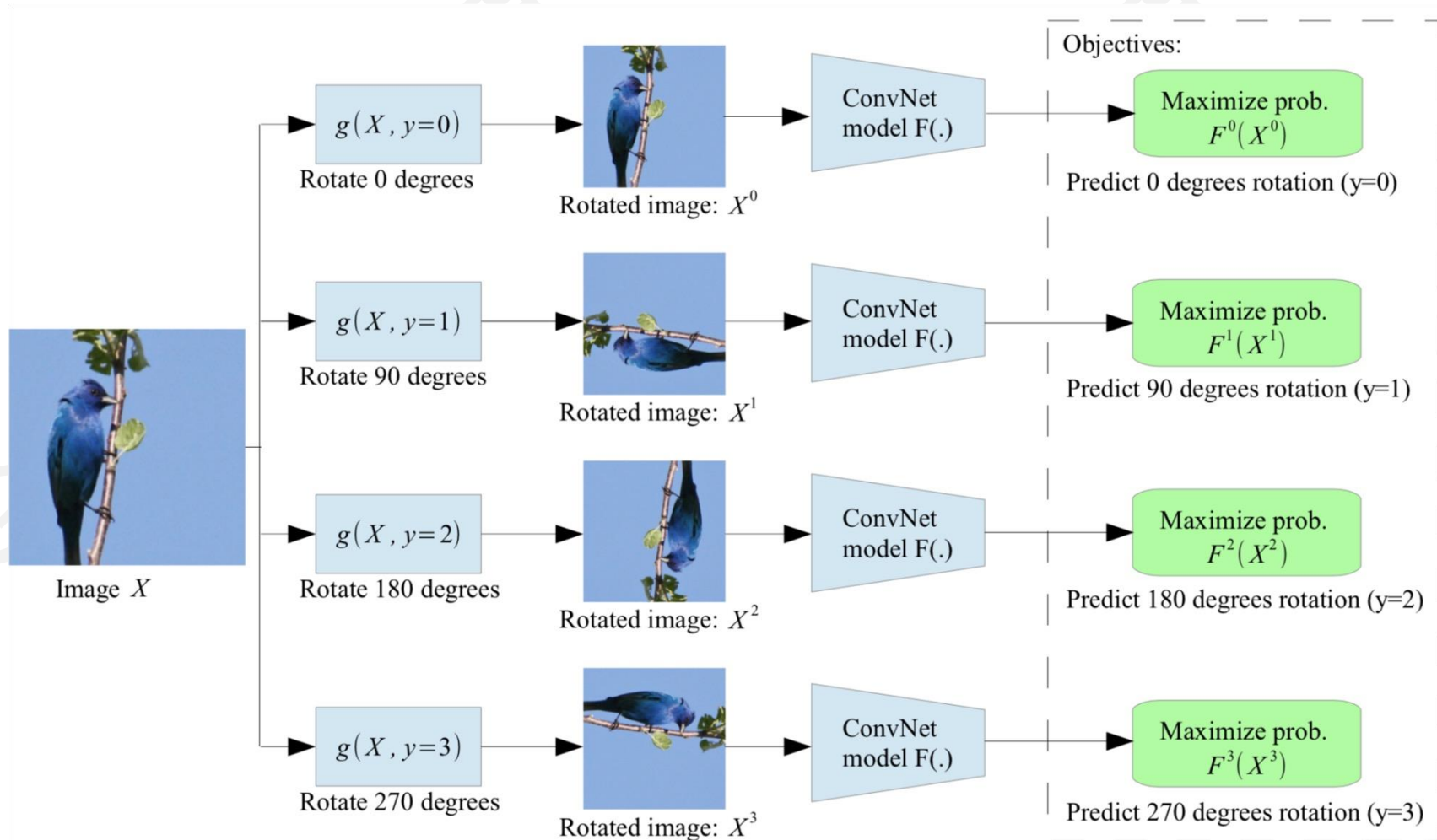


270° rotation

- 预测旋转角度
 - 通过旋转整个输入图像进行自监督学习
 - 该模型学习预测旋转的类别 (4 种旋转类别)



■ 预测旋转角度



■ 预测旋转角度

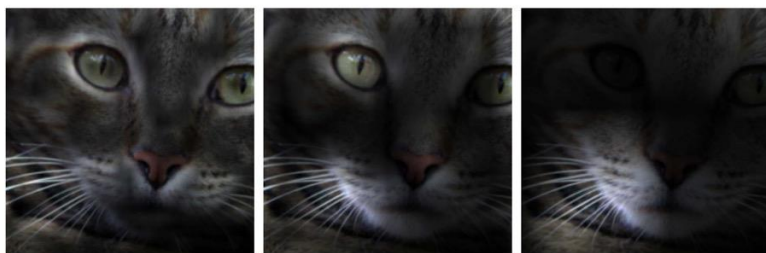
	Classification (%mAP)		Detection (%mAP)	Segmentation (%mIoU)
Trained layers	fc6-8	all	all	all
ImageNet labels	78.9	79.9	56.8	48.0
Random		53.3	43.4	19.8
Random rescaled Krähenbühl et al. (2015)	39.2	56.6	45.6	32.6
Egomotion (Agrawal et al., 2015)	31.0	54.2	43.9	
Context Encoders (Pathak et al., 2016b)	34.6	56.5	44.5	29.7
Tracking (Wang & Gupta, 2015)	55.6	63.1	47.4	
Context (Doersch et al., 2015)	55.1	65.3	51.1	
Colorization (Zhang et al., 2016a)	61.5	65.6	46.9	35.6
BIGAN (Donahue et al., 2016)	52.3	60.1	46.9	34.9
Jigsaw Puzzles (Noroozi & Favaro, 2016)	-	67.6	53.2	37.6
NAT (Bojanowski & Joulin, 2017)	56.7	65.3	49.4	
Split-Brain (Zhang et al., 2016b)	63.0	67.1	46.7	36.0
ColorProxy (Larsson et al., 2017)		65.9		38.4
Counting (Noroozi et al., 2017)	-	67.7	51.4	36.6
(Ours) RotNet	70.87	72.97	54.4	39.1

Pretrained with full ImageNet supervision

No pretraining

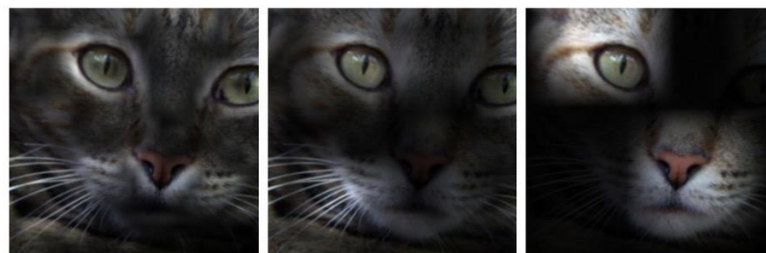
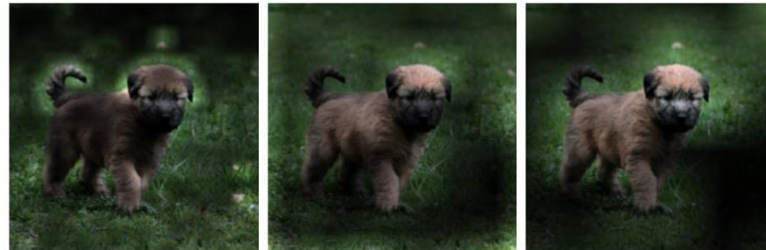
自监督学习方法
+ downstream task 数据集微调

■ 可视化学习到的特征注意力



Conv1 27×27 Conv3 13×13 Conv5 6×6

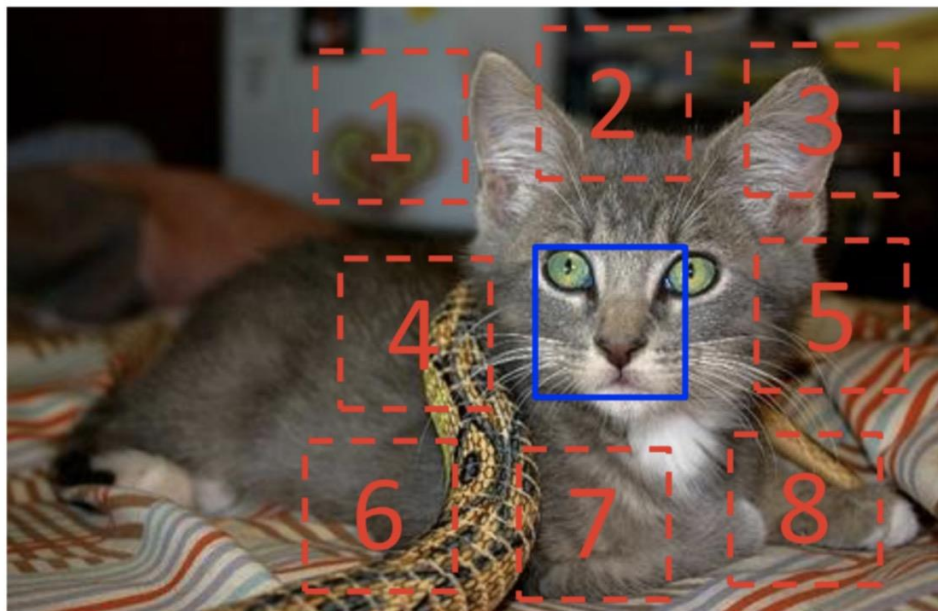
(a) Attention maps of supervised model



Conv1 27×27 Conv3 13×13 Conv5 6×6

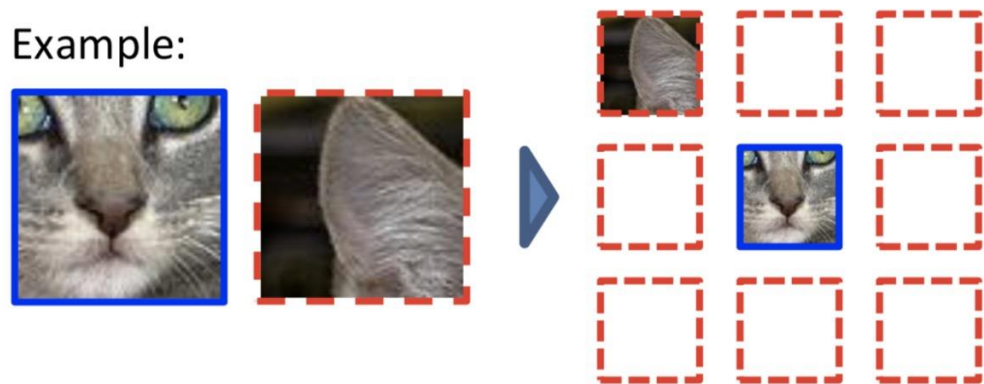
(b) Attention maps of our self-supervised model

■ 预测图像块位置



$$X = (\text{cat face}, \text{cat ear}); Y = 3$$

Example:



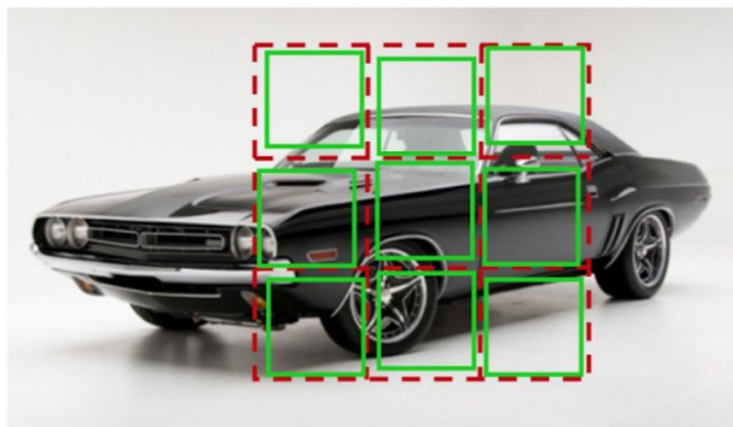
Question 1:



Question 2:



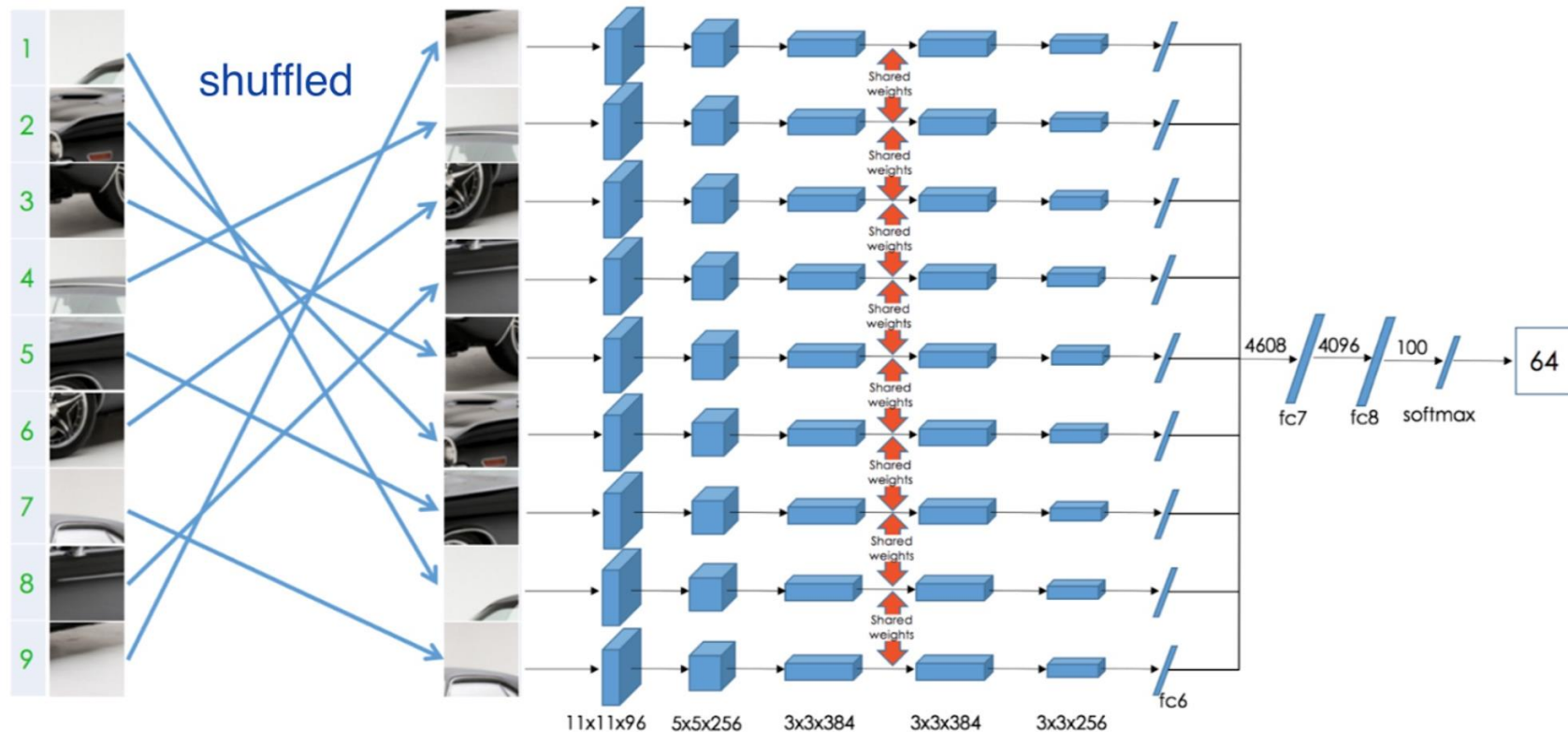
■ 预测图像拼图



Permutation Set

index	permutation
64	9,4,6,8,3,2,5,1,7

Reorder patches according to the selected permutation

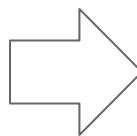


■ 将学习到的特征转移到监督学习

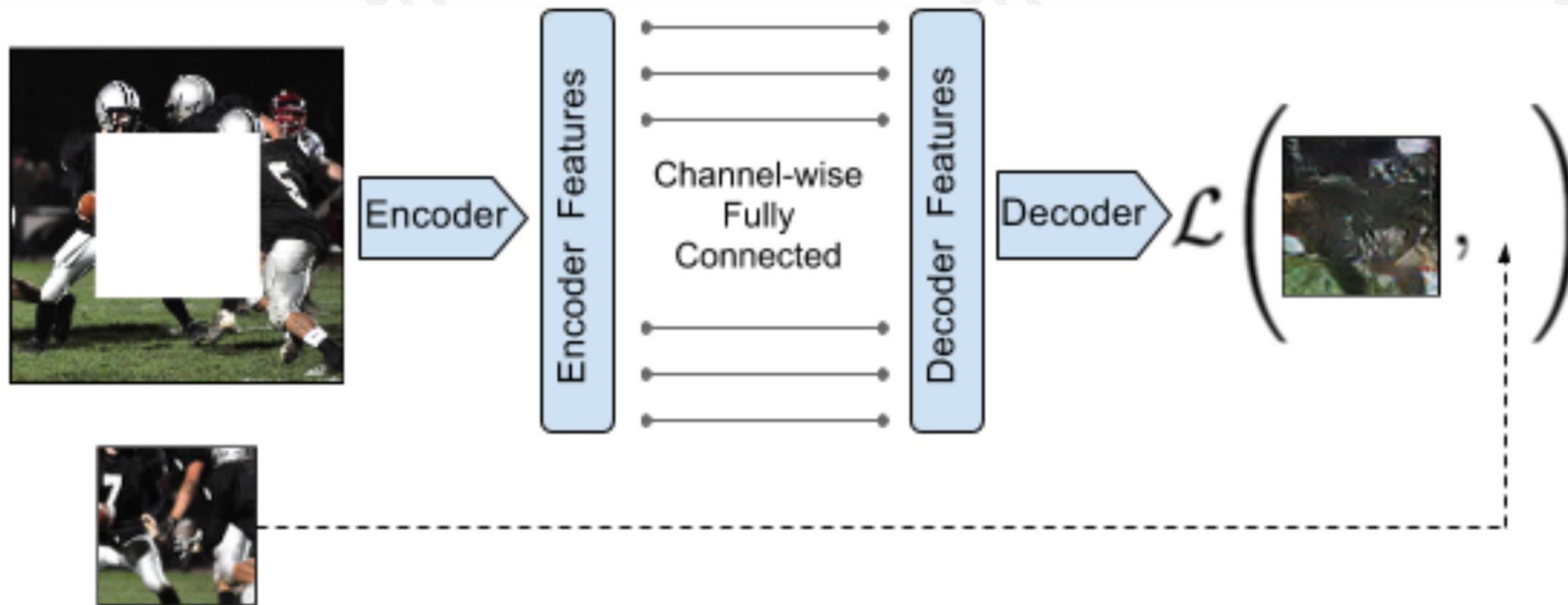
Table 1: Results on PASCAL VOC 2007 Detection and Classification. The results of the other methods are taken from Pathak *et al.* [30].

Method	Pretraining time	Supervision	Classification	Detection	Segmentation
Krizhevsky <i>et al.</i> [25]	3 days	1000 class labels	78.2%	56.8%	48.0%
Wang and Gupta[39]	1 week	motion	58.4%	44.0%	-
Doersch <i>et al.</i> [10]	4 weeks	context	55.3%	46.6%	-
Pathak <i>et al.</i> [30]	14 hours	context	56.5%	44.5%	29.7%
Ours	2.5 days	context	67.6%	53.2%	37.6%

■ 预测缺失的像素 (Inpainting)



■ 通过图像重建进行 Inpainting



■ Inpainting 效果评估



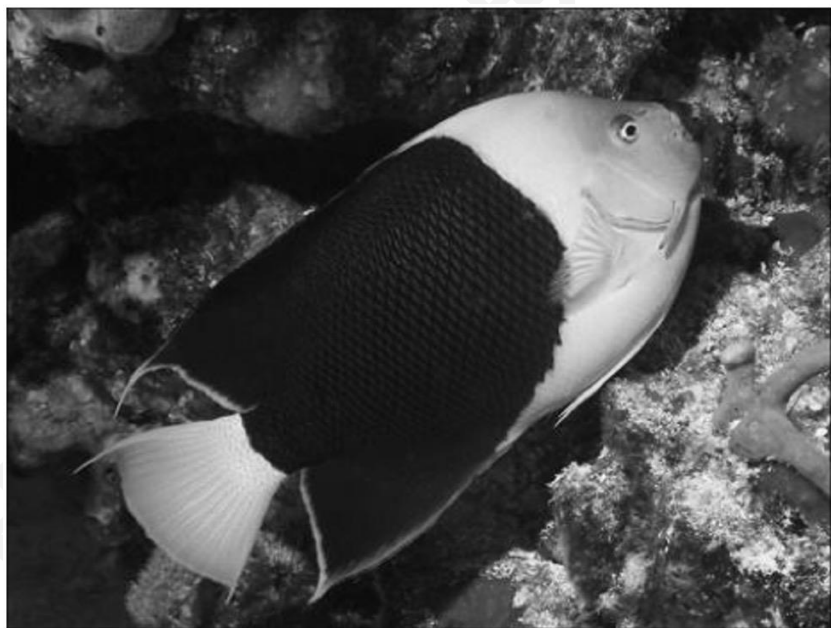
Input (context)

reconstruction

■ 将学习到的特征转移到监督学习

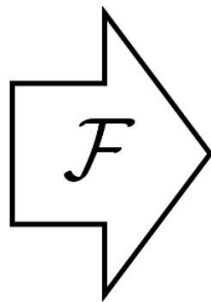
Pretraining Method	Supervision	Pretraining time	Classification	Detection	Segmentation
ImageNet [26]	1000 class labels	3 days	78.2%	56.8%	48.0%
Random Gaussian	initialization	< 1 minute	53.3%	43.4%	19.8%
Autoencoder	-	14 hours	53.8%	41.9%	25.2%
Agrawal <i>et al.</i> [1]	egomotion	10 hours	52.9%	41.8%	-
Wang <i>et al.</i> [39]	motion	1 week	58.7%	47.4%	-
Doersch <i>et al.</i> [7]	relative context	4 weeks	55.3%	46.6%	-
Ours	context	14 hours	56.5%	44.5%	30.0%

■ 图像着色



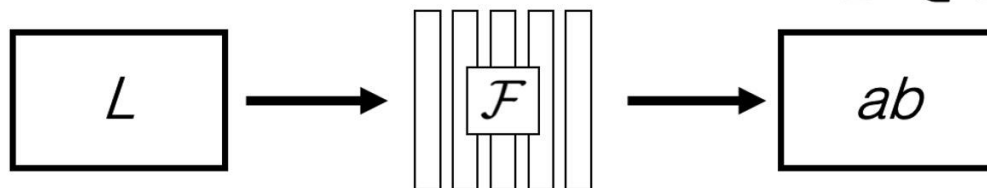
Grayscale image: L channel

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 1}$$

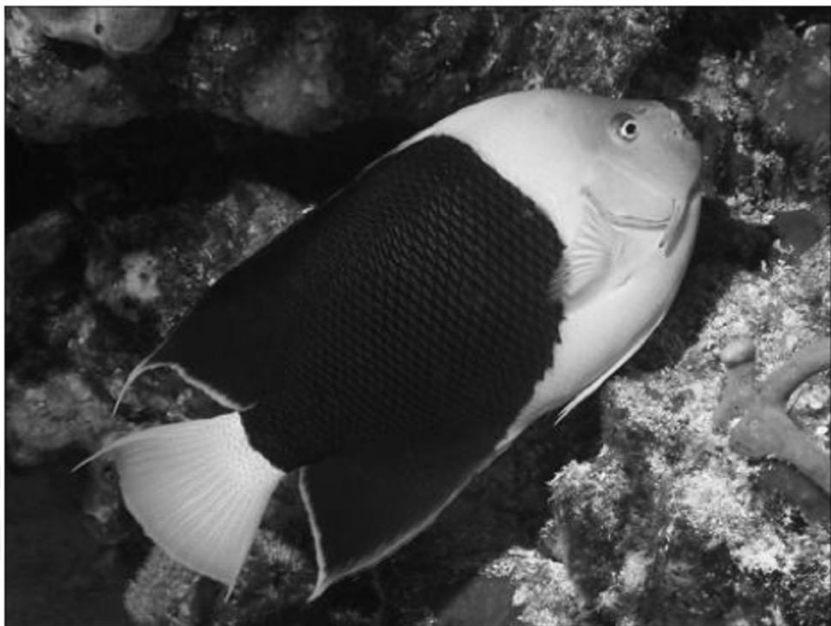


Color information: ab channels

$$\hat{\mathbf{Y}} \in \mathbb{R}^{H \times W \times 2}$$



■ 图像着色



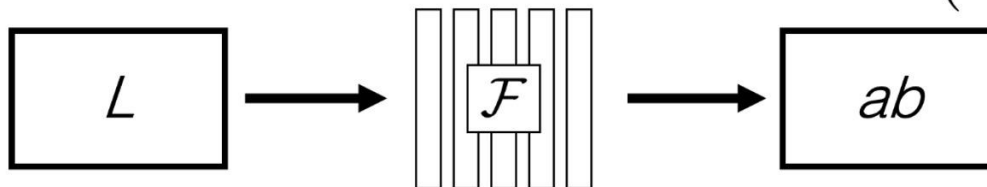
Grayscale image: L channel

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 1}$$

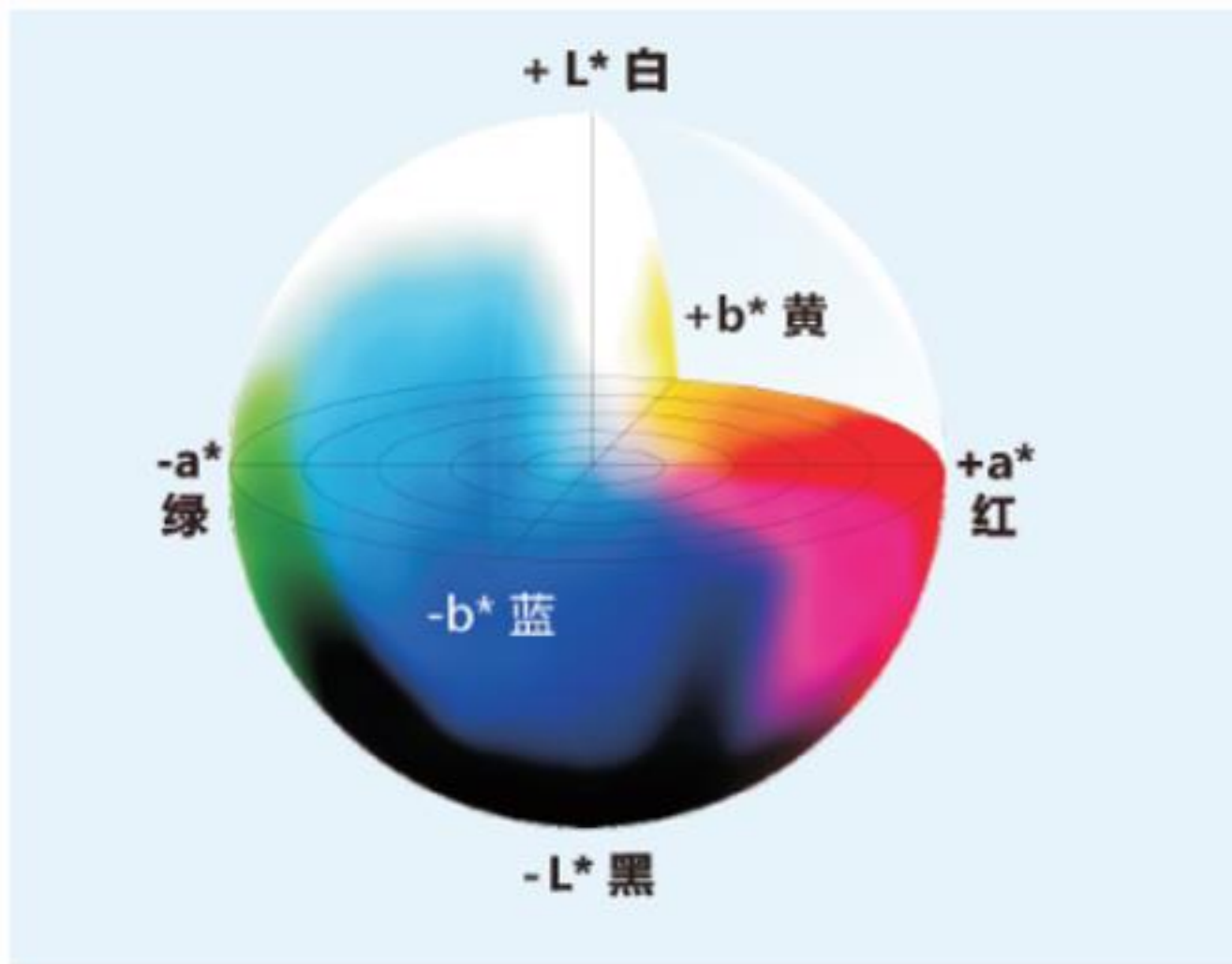


Concatenate (L, ab) channels

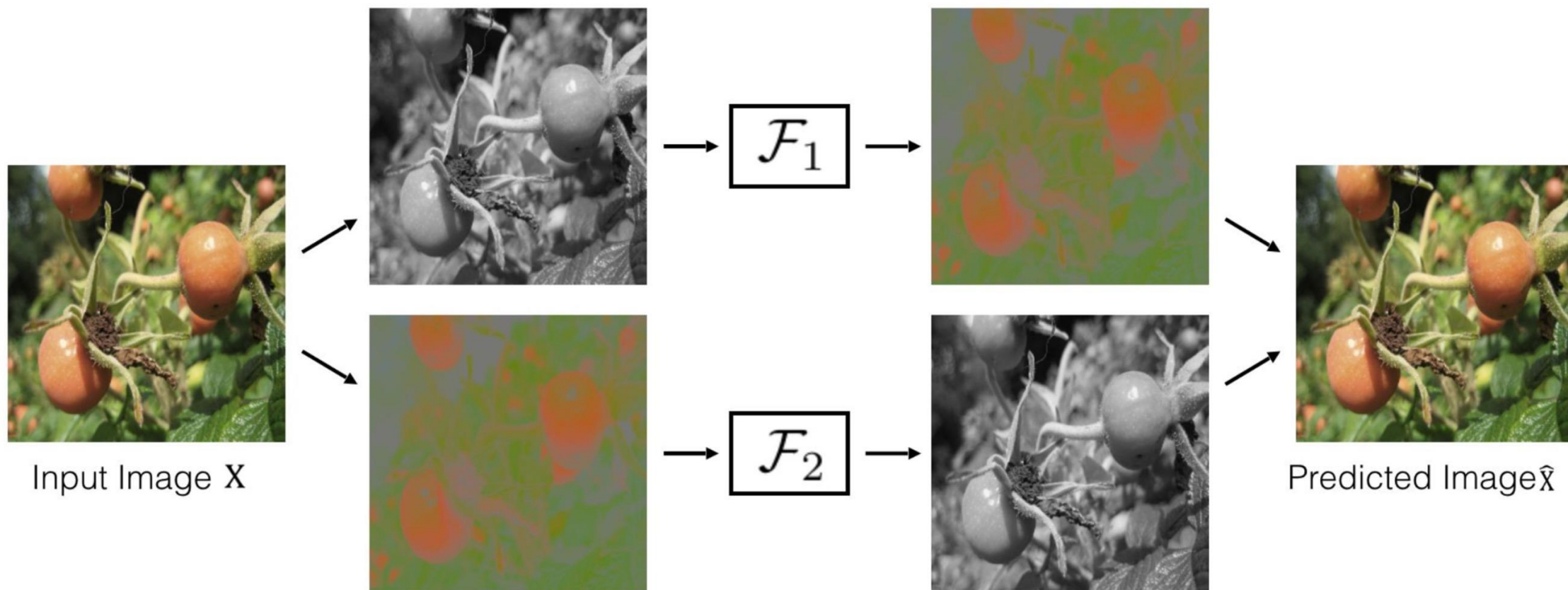
$$(\mathbf{X}, \hat{\mathbf{Y}})$$



■ 图像着色

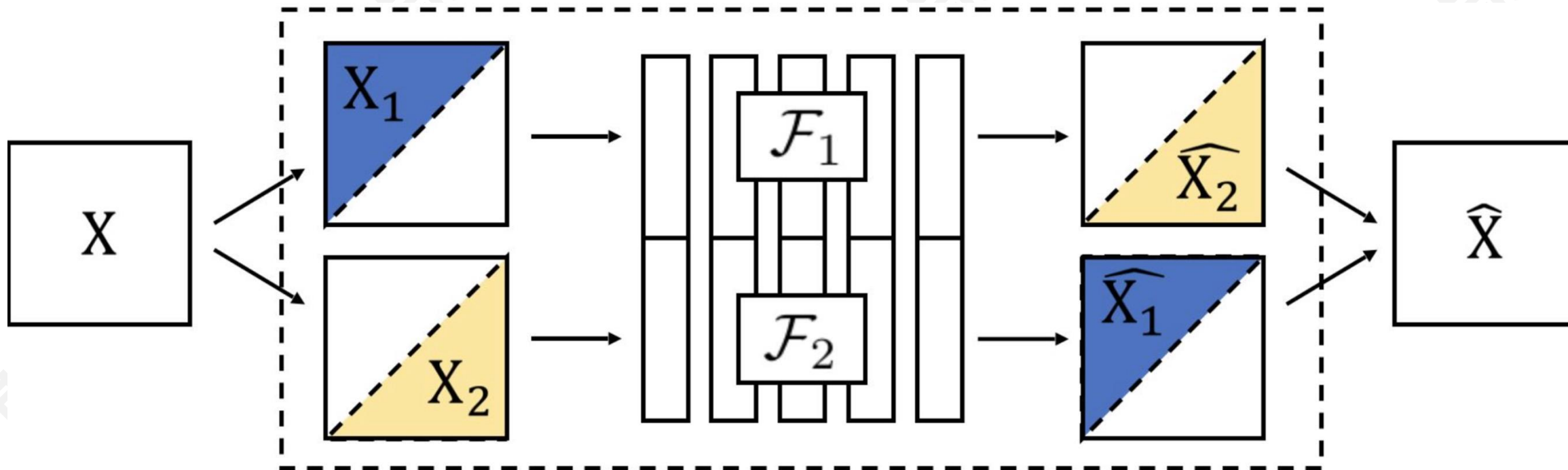


■ 从图像着色中学习特征: Split-brain Autoencoder



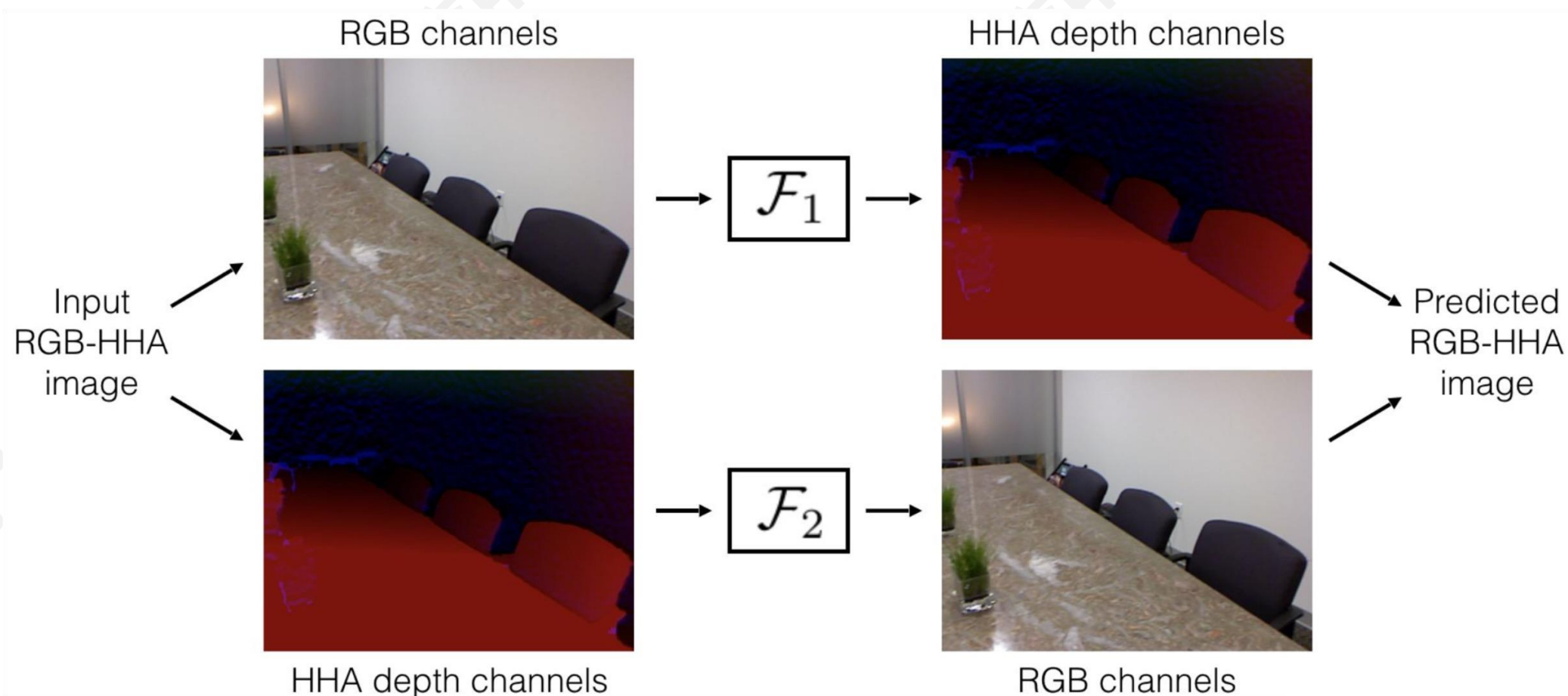
■ 从图像着色中学习特征: Split-brain Autoencoder

■ Idea: 跨通道预测

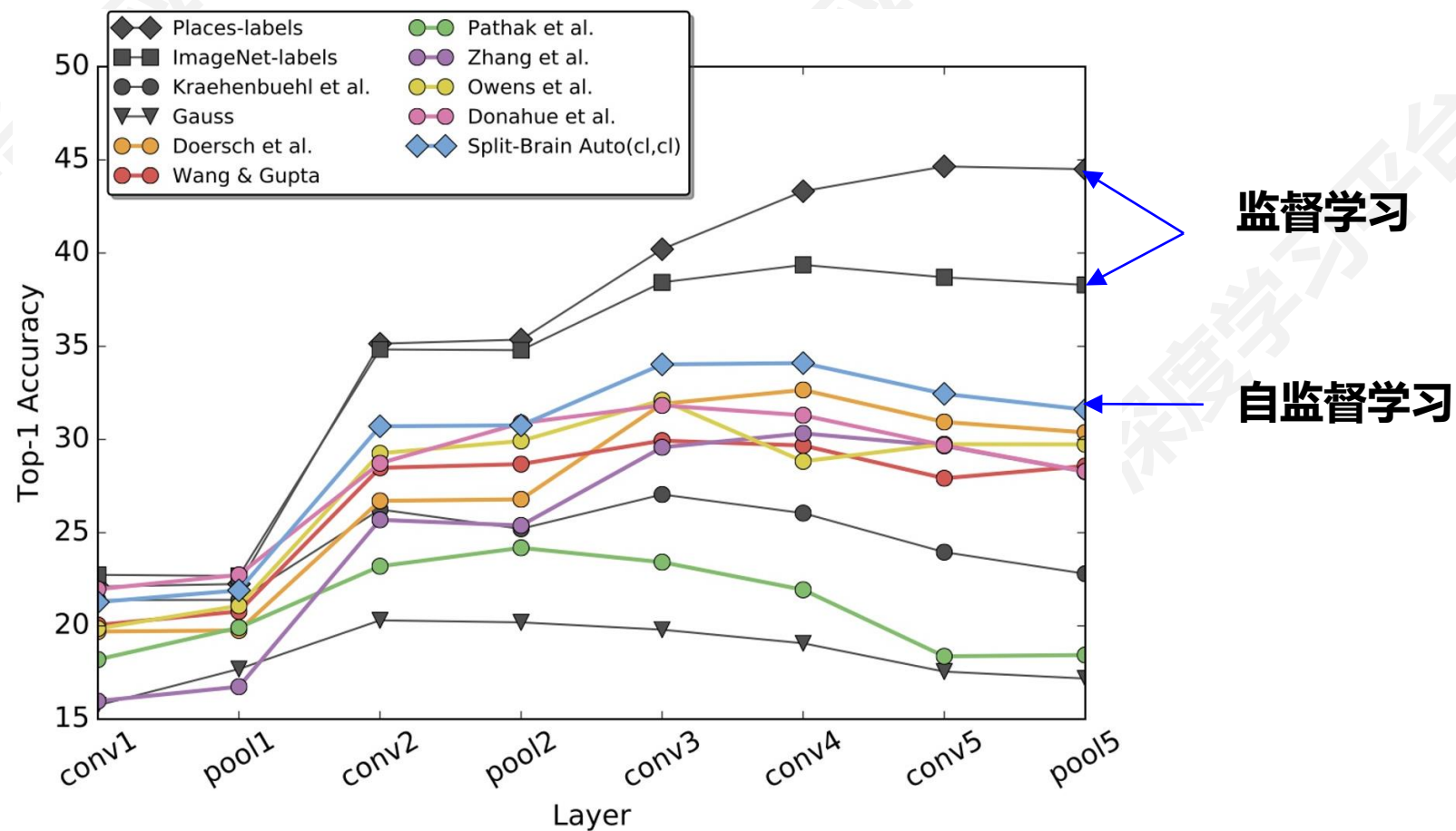


Split-Brain Autoencoder

■ 从图像着色中学习特征: Split-brain Autoencoder



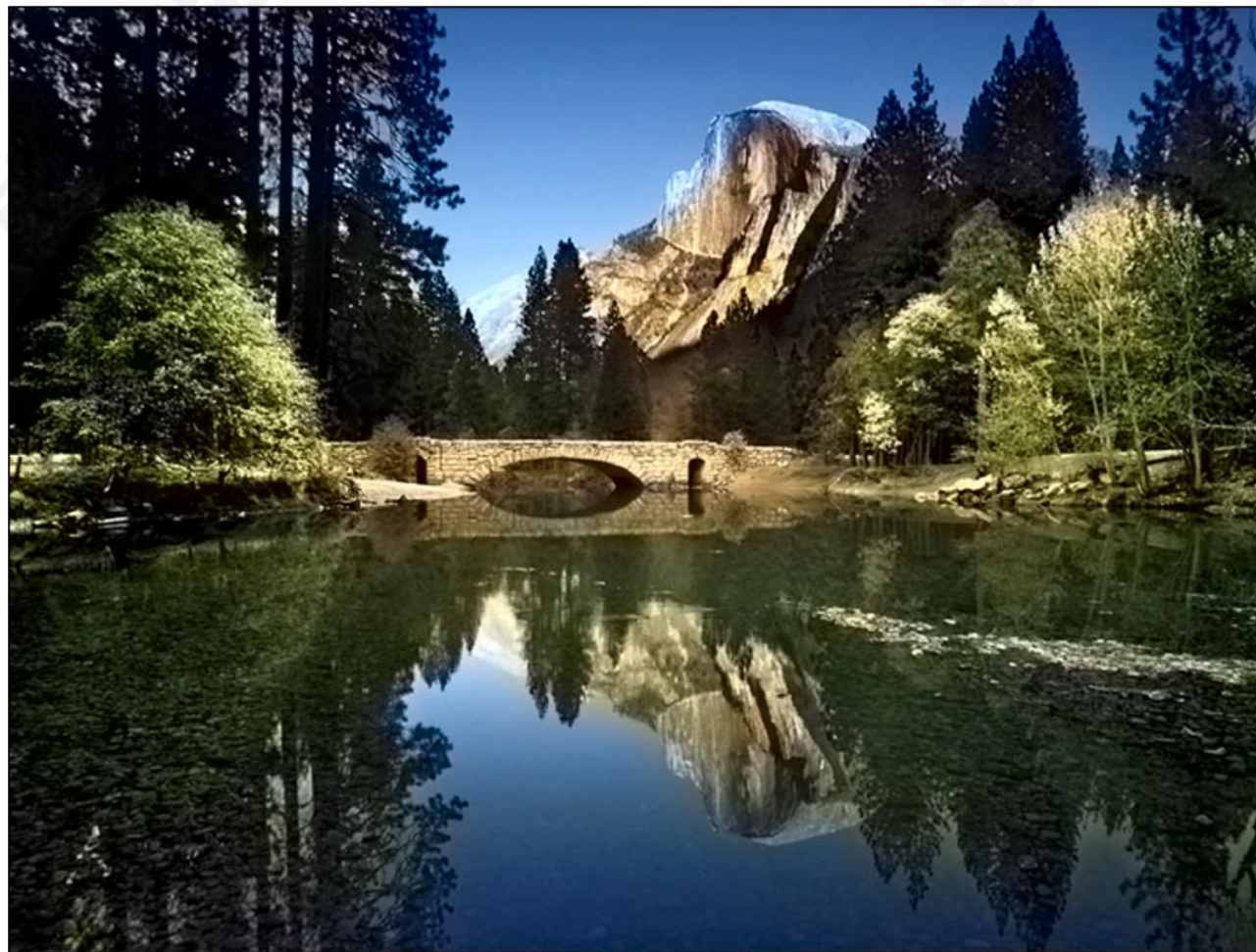
■ 将学习到的特征转移到监督学习



■ 图像着色



■ 图像着色



■ 视频着色

■ Idea: 建模视频中颜色的时间连贯性

参考图像



需要着色的视频帧



$t = 1$



$t = 2$

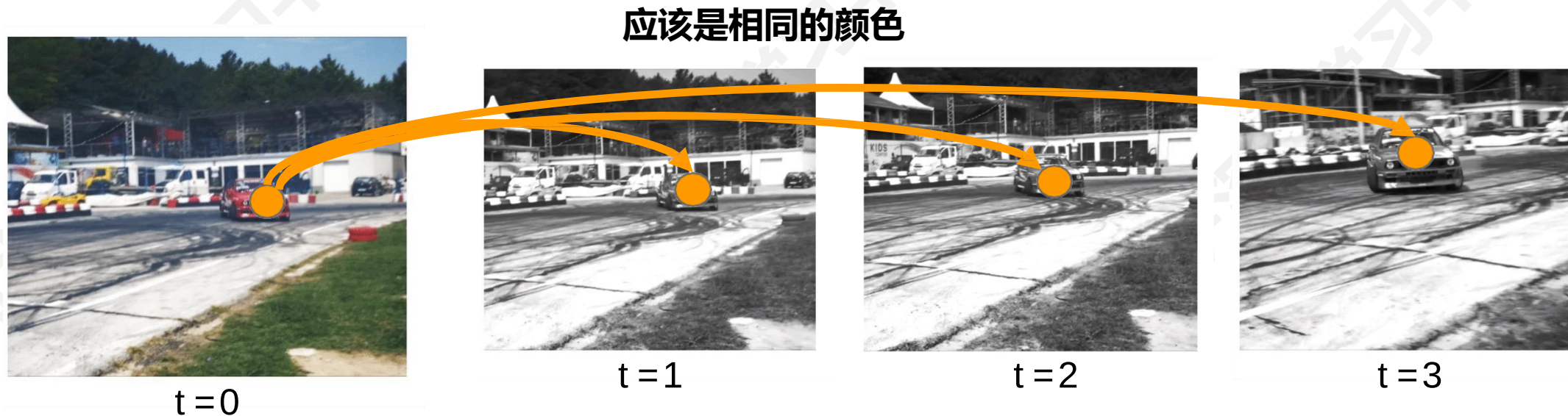


$t = 3$

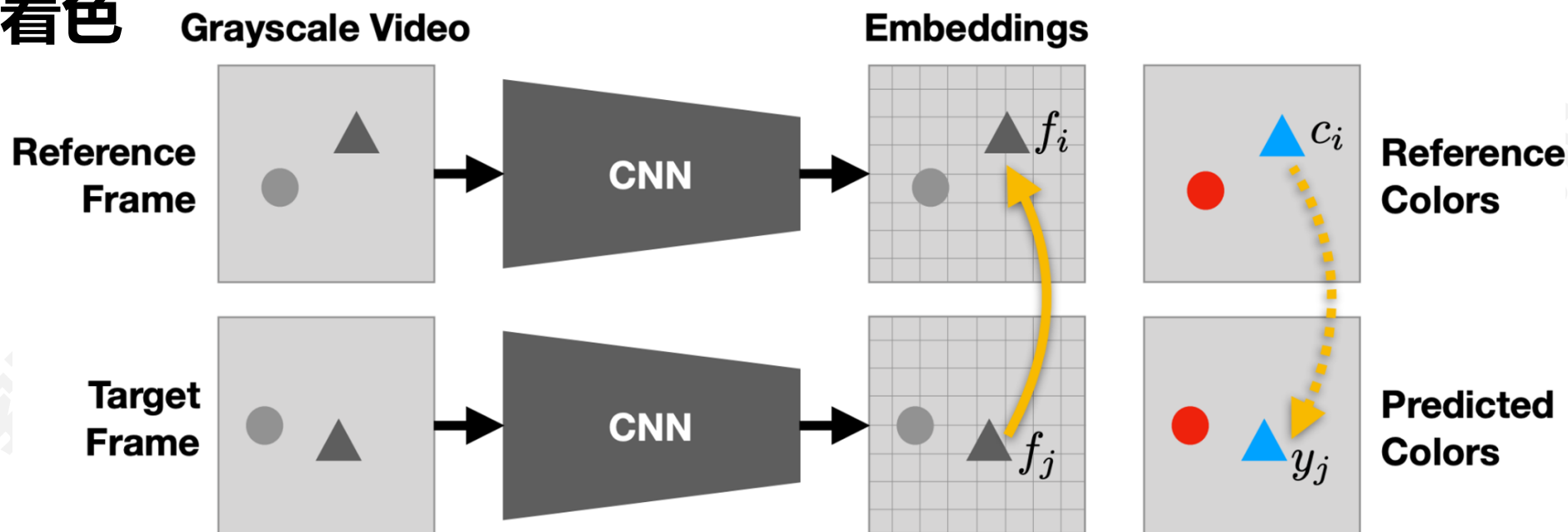
...

■ 视频着色

- 假设：可以使模型学习跟踪没有标签的区域或对象



■ 视频着色



参考图像上的注意力图

预测颜色=参考颜色的加权和

预测颜色与真实颜色之间的损失

$$A_{ij} = \frac{\exp(f_i^T f_j)}{\sum_k \exp(f_k^T f_j)}$$

$$y_j = \sum_i A_{ij} c_i$$

$$\min_{\theta} \sum_j \mathcal{L}(y_j, c_j)$$

■ 视频着色

参考图像



目标图像



预测颜色



■ 视频着色

参考图像



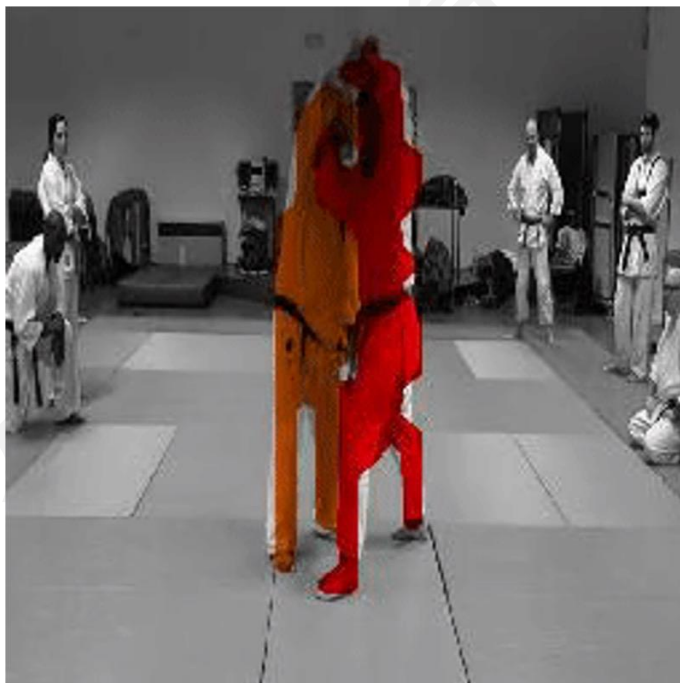
目标图像



预测颜色



- 视频着色与跟踪紧密相关
 - 利用学习到的注意力传播分割掩码



- 视频着色与跟踪紧密相关
 - 利用学习到的注意力传播姿势关键点



- **Pretext Task 侧重于“视觉常识”，例如预测旋转、修复、重新排列和着色**
- **模型需要学习自然图像的良好特征，例如对象类别的语义表示，以解决 Pretext Task**
- **我们通常不关心这些 Pretext Task 的性能，而是关心学习到的特征对 Downstream Task（分类、检测、分割）的效果**

- Pretext Task 学到的表征可能与特定的 Pretext Task 有关
- 我们需要一个更通用的任务

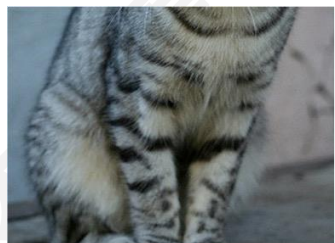
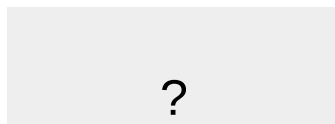
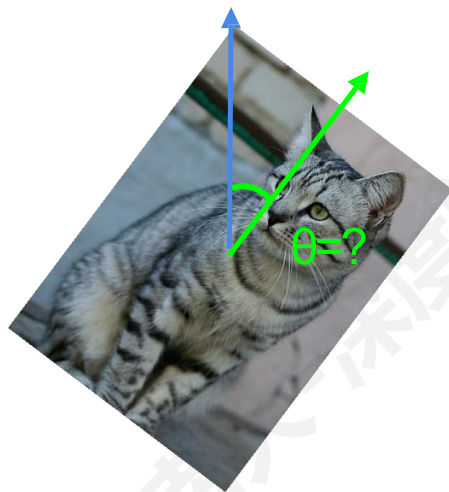


image completion



rotation prediction

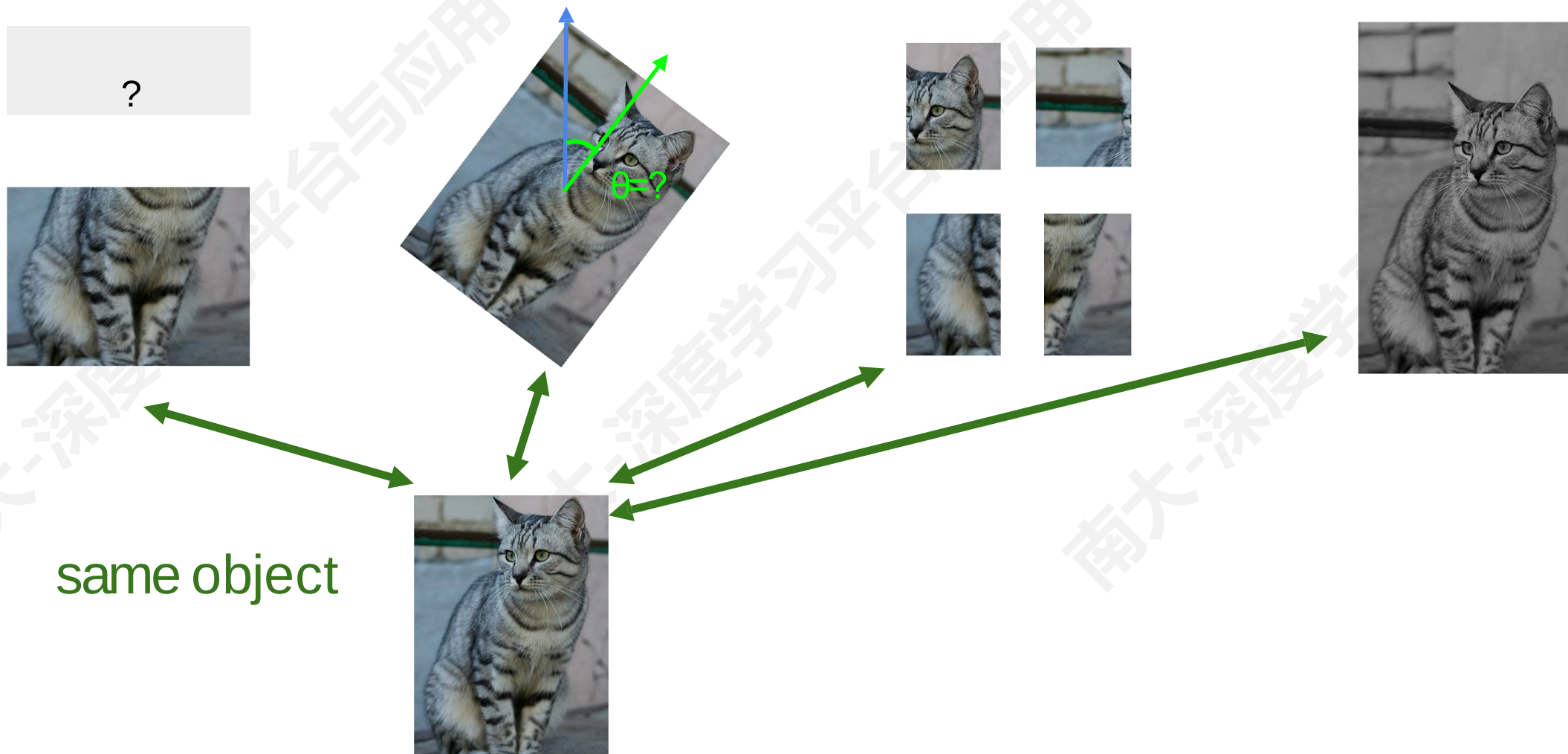


“jigsaw puzzle”

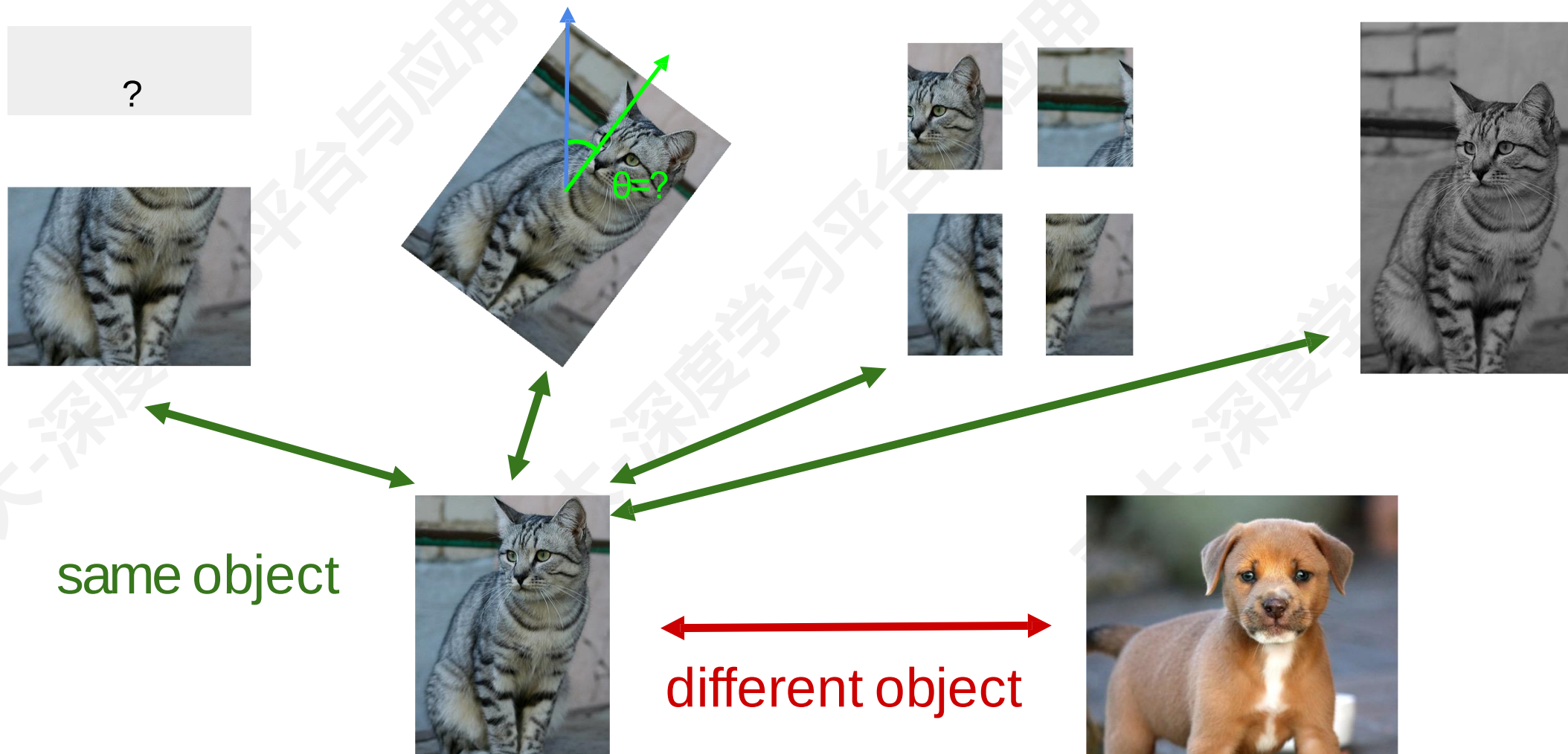


colorization

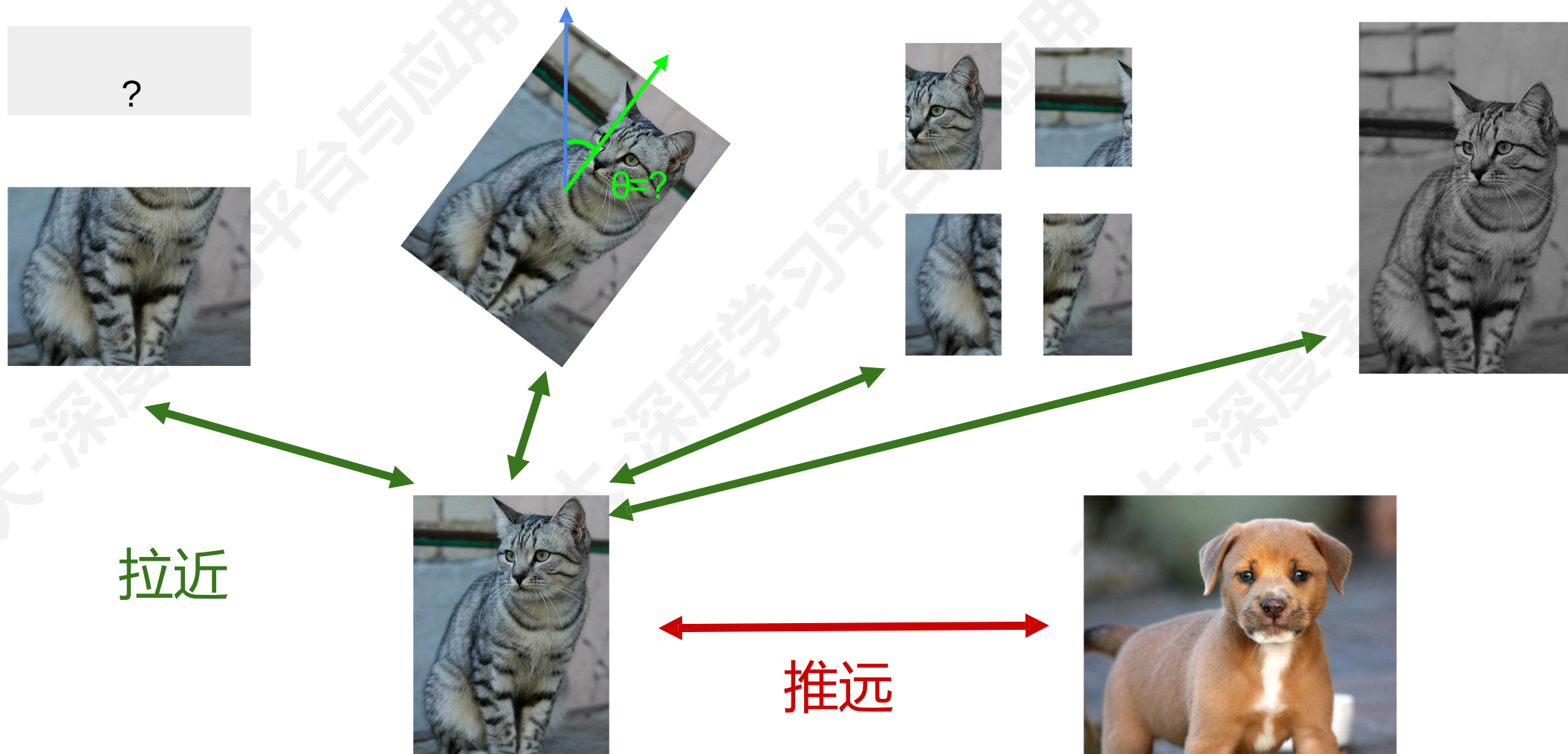
■ 更通用的 Pretext Task?



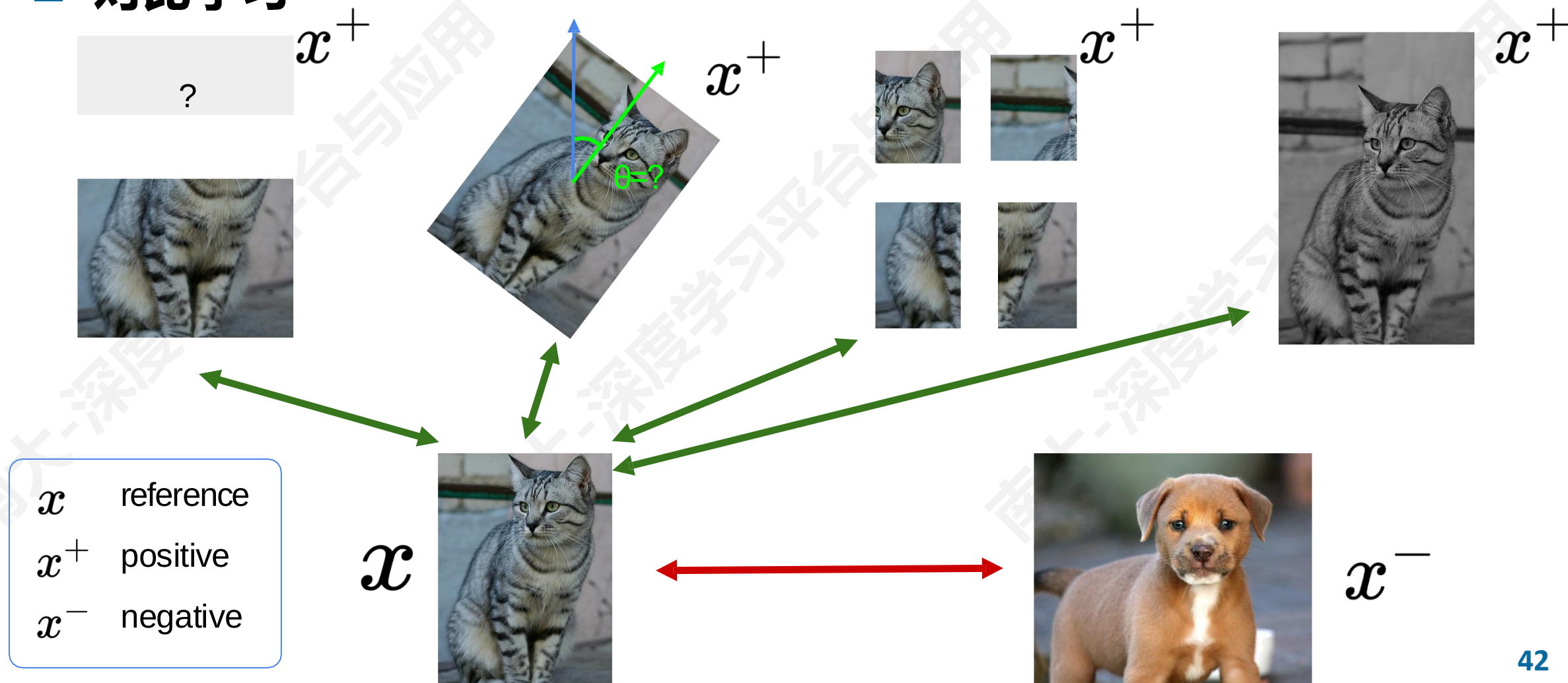
■ 更通用的 Pretext Task?



■ 对比学习



■ 对比学习



■ 对比学习的形式化定义

$$\text{score}(f(x), f(x^+)) \gg \text{score}(f(x), f(x^-))$$

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

■ 对比学习的形式化定义

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}}{\underbrace{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}} \right]$$



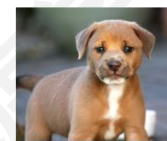
x



x^+



x



x_1^-



x_2^-



x_3^-

...

N路softmax分类器的交叉熵损失!
即, 学会从N个样本中找出正样本

■ 对比学习的形式化定义

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

■ 通常称为 InfoNCE Loss

■ SimCLR: A Simple Framework for Contrastive Learning

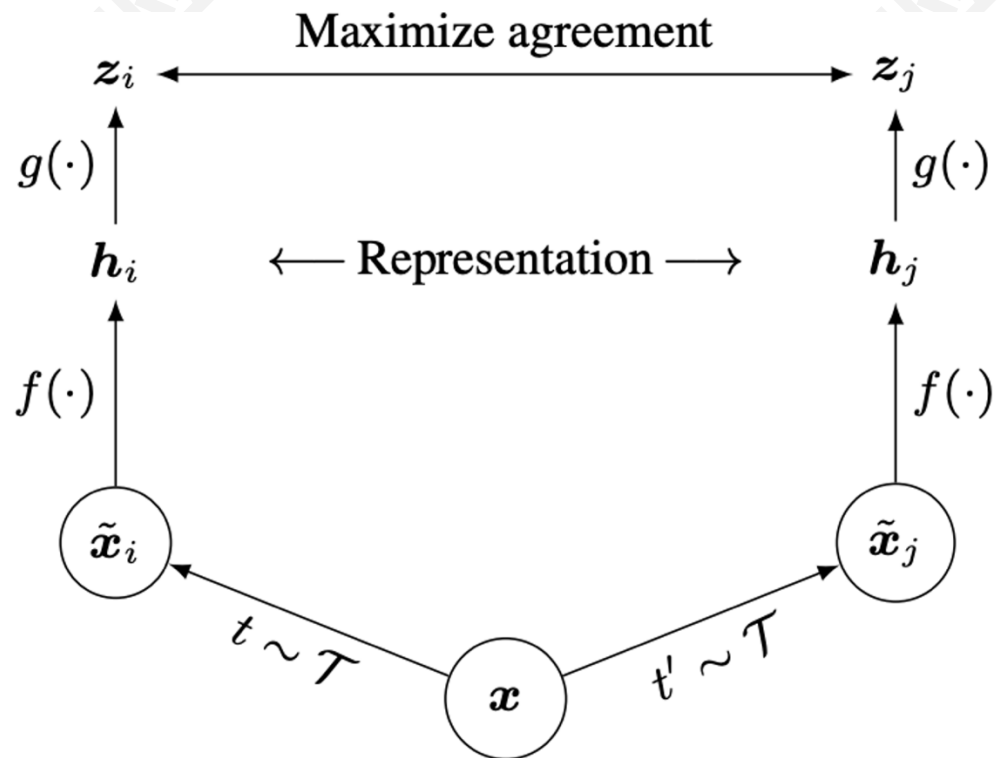
■ 余弦相似度: $s(u, v) = \frac{u^T v}{\|u\| \|v\|}$

■ 通过数据增强生成正样本

■ 随机裁剪

■ 随机颜色失真

■ 随机模糊



■ SimCLR: 数据增强生成正样本



(a) Original



(b) Crop and resize



(c) Crop, resize (and flip)



(d) Color distort. (drop)



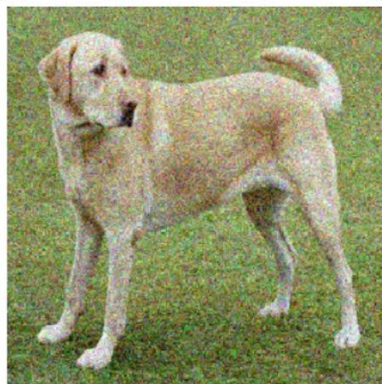
(e) Color distort. (jitter)



(f) Rotate $\{90^\circ, 180^\circ, 270^\circ\}$



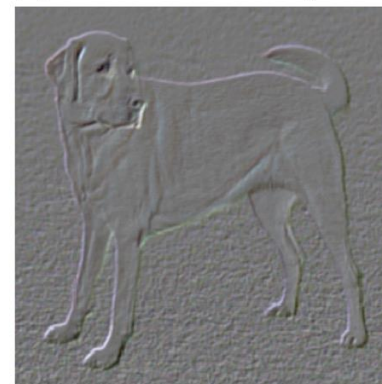
(g) Cutout



(h) Gaussian noise



(i) Gaussian blur



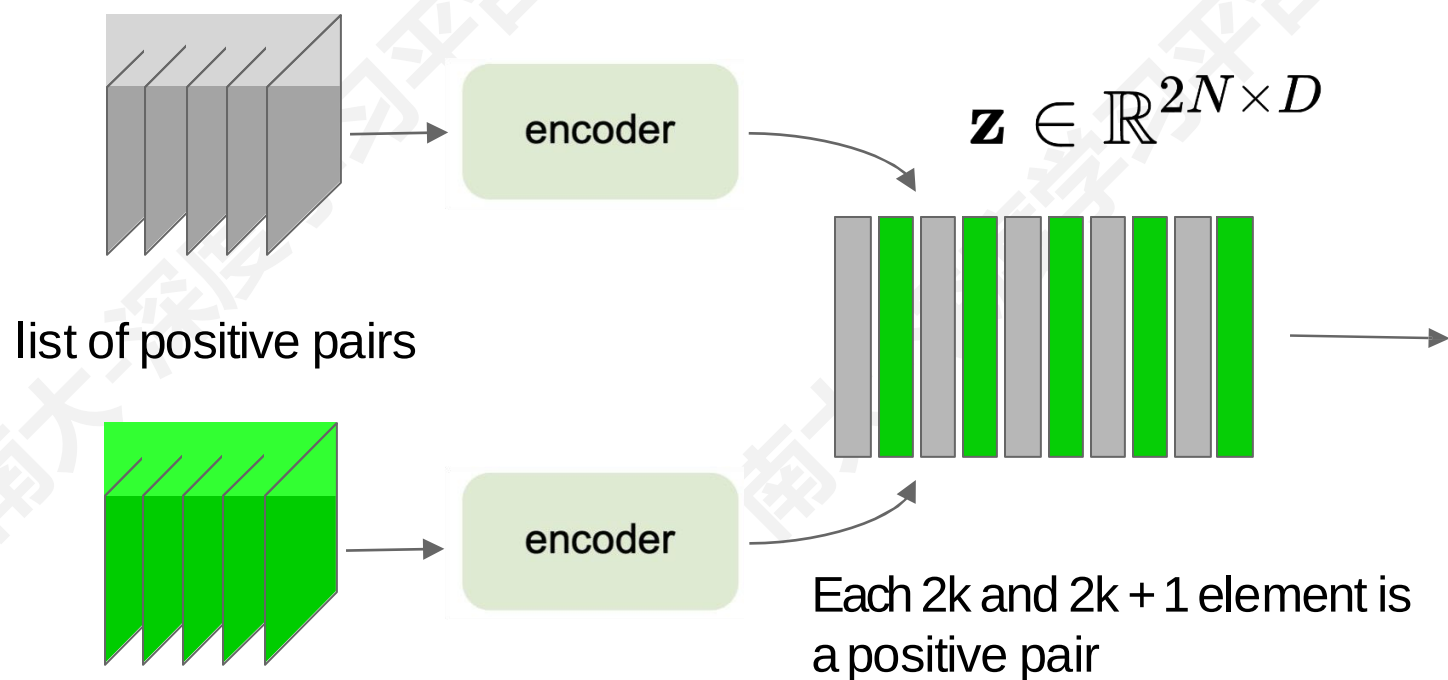
(j) Sobel filtering

■ SimCLR

Algorithm 1 SimCLR's main learning algorithm.

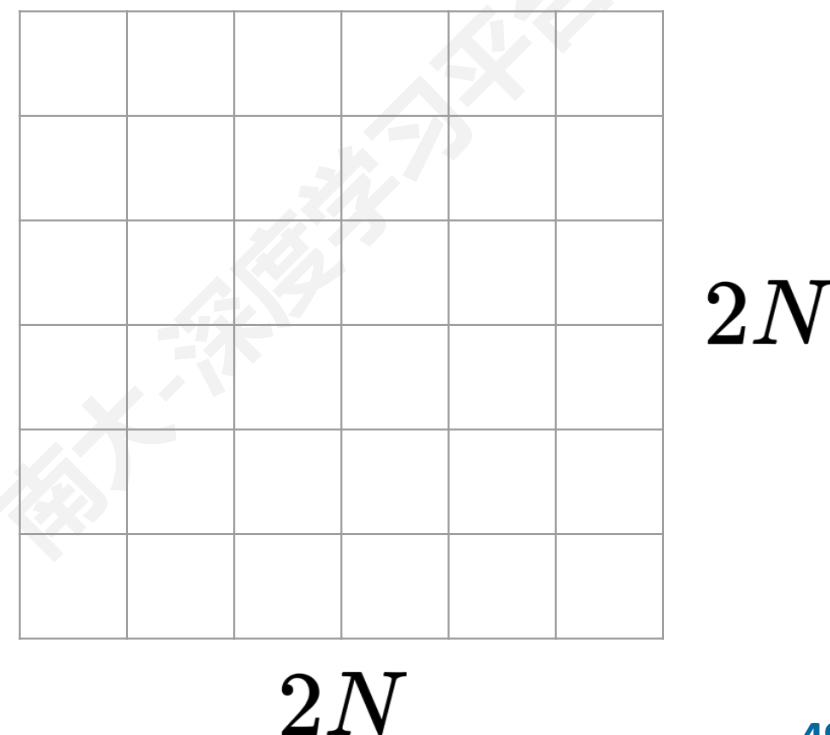
input: batch size N , constant τ , structure of f , g , $\mathcal{T}$.
for sampled minibatch $\{\mathbf{x}_k\}_{k=1}^N$ **do**
 for all $k \in \{1, \dots, N\}$ **do**
 draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
 # the first augmentation
 $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$
 $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$ # representation
 $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$ # projection
 # the second augmentation
 $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$
 $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$ # representation
 $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$ # projection
 end for
 for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**
 $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ # pairwise similarity
 end for
 define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
 $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
 update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

■ SimCLR: mini-batch training

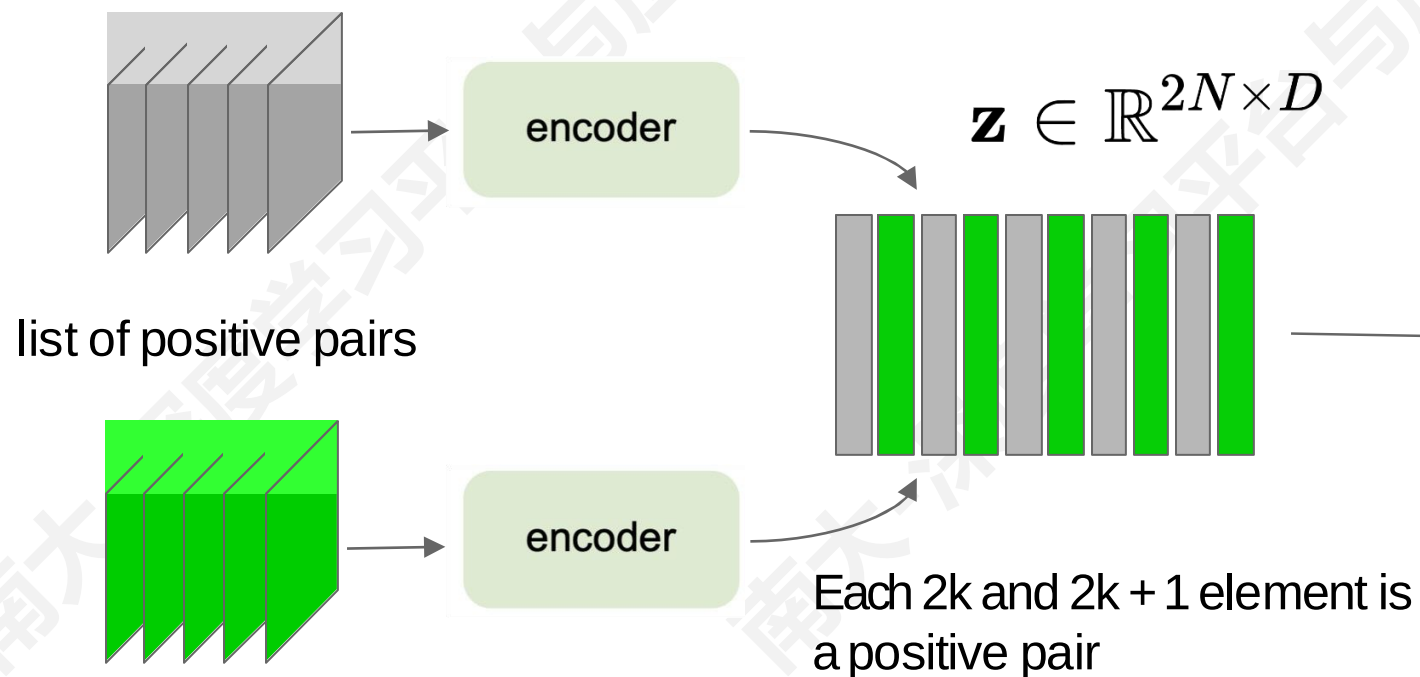


$$s_{i,j} = \frac{z_i^T z_j}{\|z_i\| \|z_j\|}$$

"Affinity matrix"

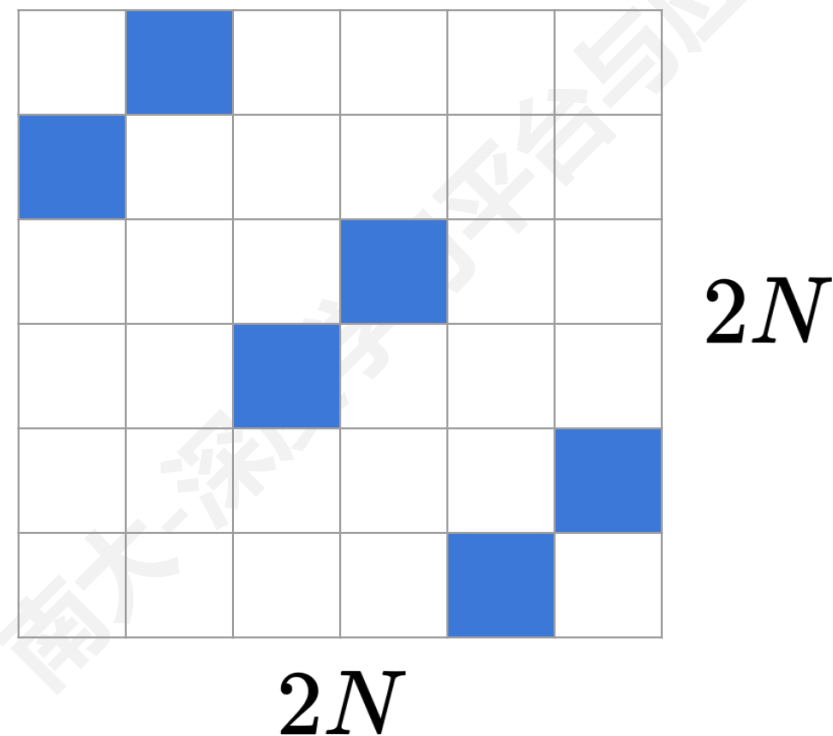


■ SimCLR: mini-batch training



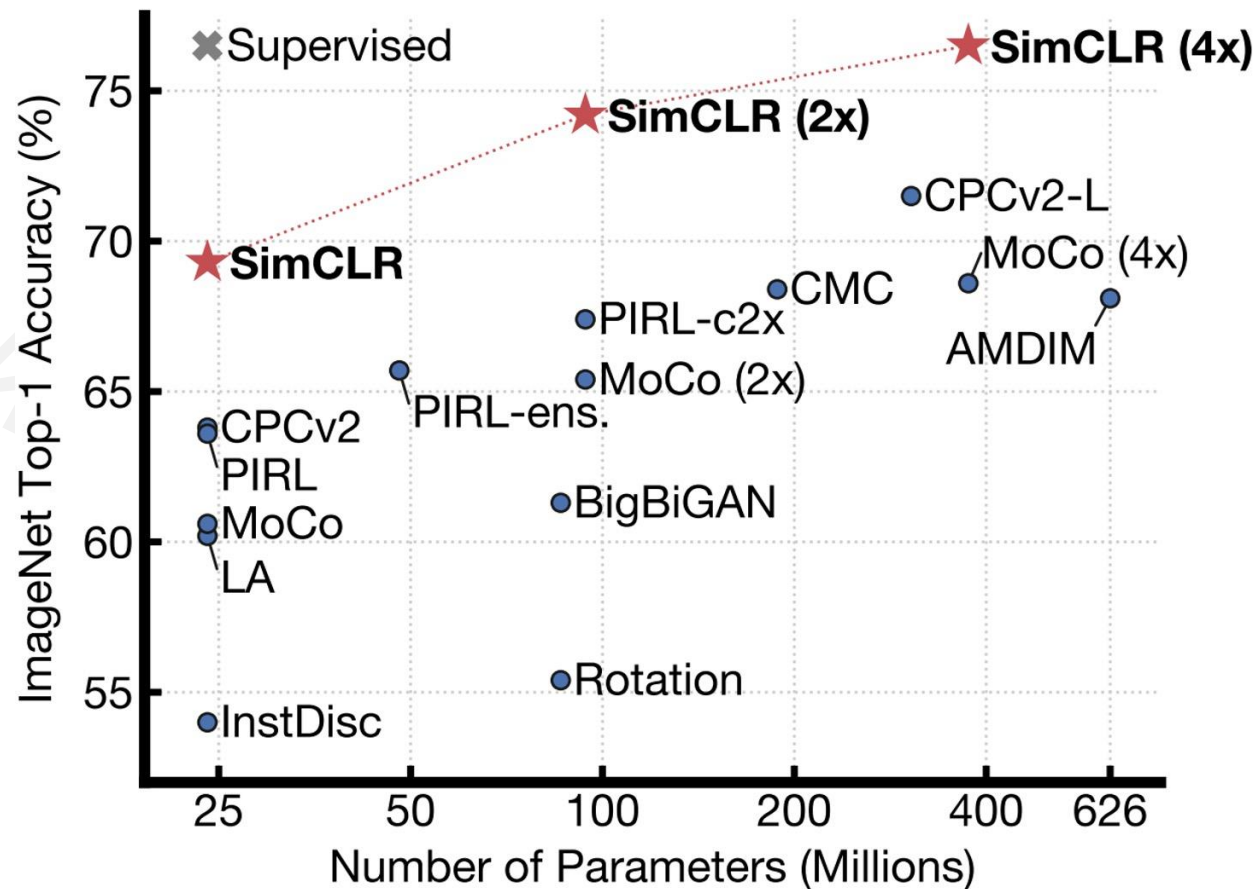
$$s_{i,j} = \frac{z_i^T z_j}{\|z_i\| \|z_j\|}$$

"Affinity matrix"



 = classification label for each row

- 基于 SimCLR 特征训练线性分类器
- 使用 SimCLR 在无标签的 ImageNet 数据集上训练特征编码器（整个训练集）
- 冻结特征编码器，在有标签数据上训练线性分类器

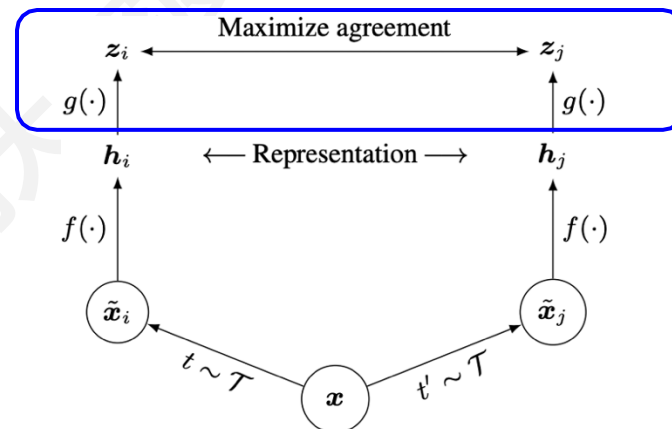
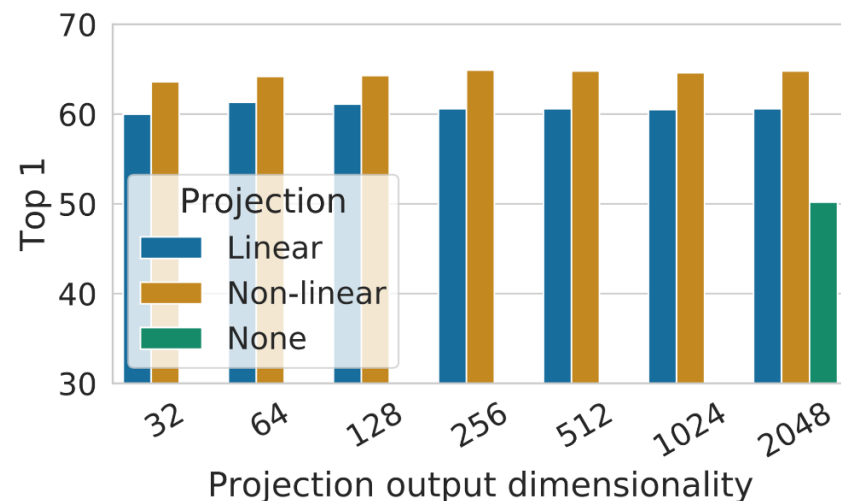


- 基于 SimCLR 特征训练线性分类器
- 使用 SimCLR 在无标签的 ImageNet 数据集上训练特征编码器（整个训练集）
- 使用 ImageNet 上1%/10% 的有标签数据微调编码器

Method	Architecture	Label fraction	
		1%	10%
Top 5			
Supervised baseline	ResNet-50	48.4	80.4
<i>Methods using other label-propagation:</i>			
Pseudo-label	ResNet-50	51.6	82.4
VAT+Entropy Min.	ResNet-50	47.0	83.4
UDA (w. RandAug)	ResNet-50	-	88.5
FixMatch (w. RandAug)	ResNet-50	-	89.1
S4L (Rot+VAT+En. M.)	ResNet-50 (4×)	-	91.2
<i>Methods using representation learning only:</i>			
InstDisc	ResNet-50	39.2	77.4
BigBiGAN	RevNet-50 (4×)	55.2	78.8
PIRL	ResNet-50	57.2	83.8
CPC v2	ResNet-161(*)	77.9	91.2
SimCLR (ours)	ResNet-50	75.5	87.8
SimCLR (ours)	ResNet-50 (2×)	83.0	91.2
SimCLR (ours)	ResNet-50 (4×)	85.8	92.6

Table 7. ImageNet accuracy of models trained with few labels.

- SimCLR 设计选择：特征投影
 - 线性/非线性投影改善了表示学习
 - 对比学习的训练目标可能会丢弃下游任务的有用信息
 - 特征空间 z 被训练为对数据转换保持不变
 - 通过利用投影头 $g(\cdot)$ ，可以在特征空间 h 中保留更多信息



- SimCLR 设计选择：大 batch
- 大型训练批量对SimCLR至关重要
- 但大批量会导致反向传播过程中占用大量内存：需要在TPU上进行分布式训练

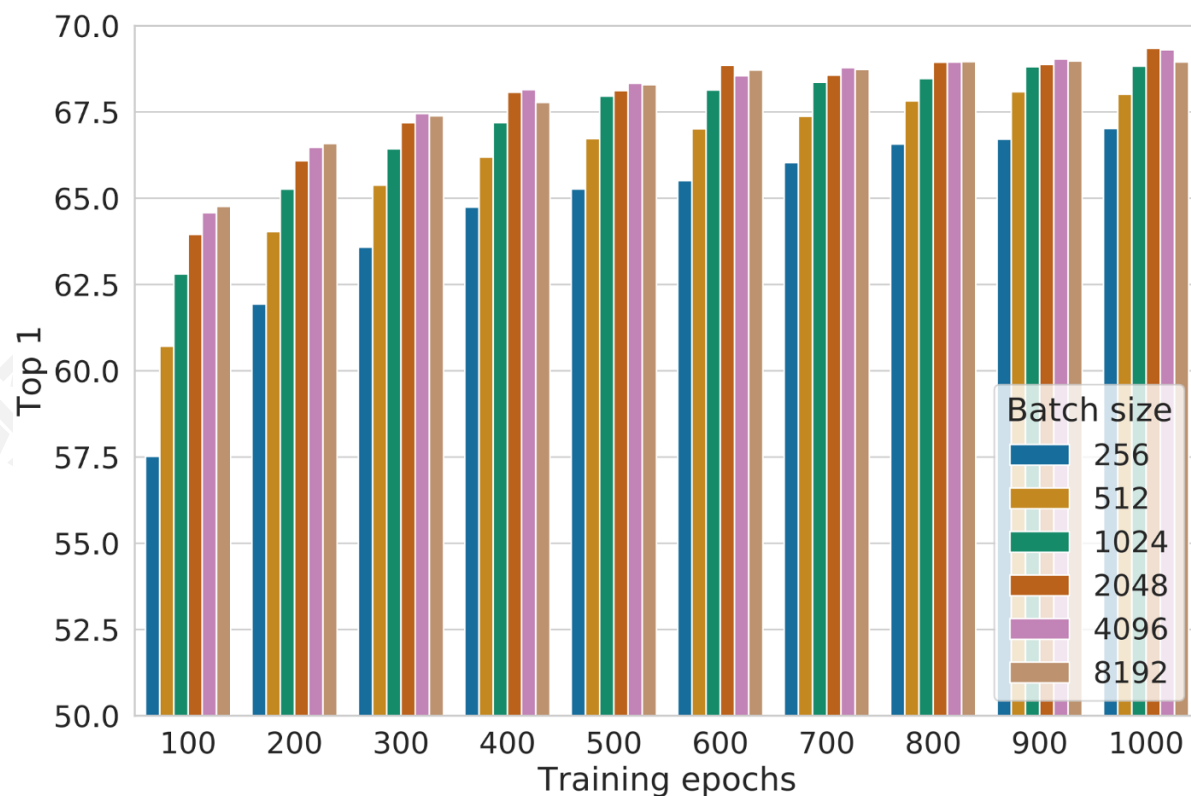


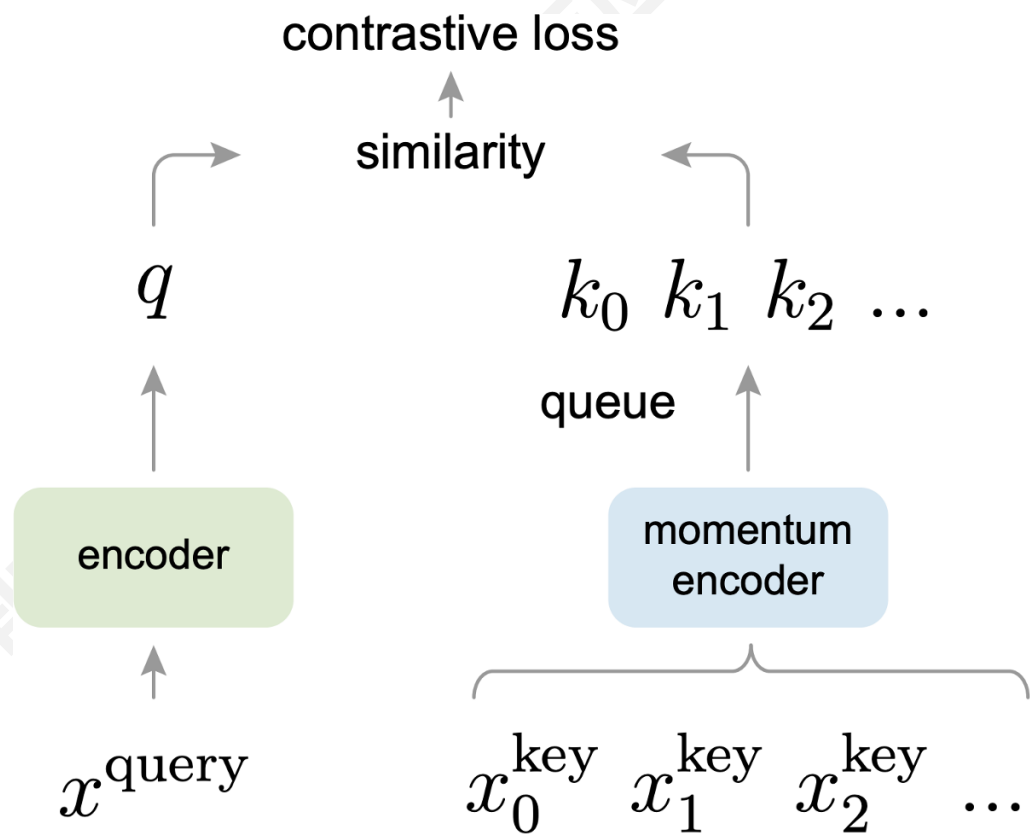
Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.¹⁰

■ Momentum Contrastive Learning (MoCo)

■ 与SimCLR的主要区别：

- 保持负样本 keys 队列
- 仅通过 queries 计算梯度并更新编码器
- 将 batch 大小与 keys 数量解耦
- 可以支持大量负样本
- Key 编码器通过动量缓慢更新

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$$



MoCo

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
    x_q = aug(x) # a randomly augmented version
    x_k = aug(x) # another randomly augmented version

    q = f_q.forward(x_q) # queries: Nx C
    k = f_k.forward(x_k) # keys: Nx C
    k = k.detach() # no gradient to keys

    # positive logits: Nx1
    l_pos = bmm(q.view(N, 1, C), k.view(N, C, 1))

    # negative logits: NxK
    l_neg = mm(q.view(N, C), queue.view(C, K))

    # logits: Nx(1+K)
    logits = cat([l_pos, l_neg], dim=1)

    # contrastive loss, Eqn. (1)
    labels = zeros(N) # positives are the 0-th
    loss = CrossEntropyLoss(logits/t, labels)

    # SGD update: query network
    loss.backward()
    update(f_q.params)

    # momentum update: key network
    f_k.params = m*f_k.params + (1-m)*f_q.params

    # update dictionary
    enqueue(queue, k) # enqueue the current minibatch
    dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

- MoCo V2
 - SimCLR 和 MoCo 的混合
 - 来自 SimCLR: 非线性投影头和强大的数据增强
 - 来自 MoCo: 动量更新了队列, 允许对大量负样本进行训练
(不需要TPU!)

Improved Baselines with Momentum Contrastive Learning

Xinlei Chen Haoqi Fan Ross Girshick Kaiming He
Facebook AI Research (FAIR)

- MoCo vs. SimCLR vs. MoCo V2
- 非线性投影头和强数据增强对于对比学习至关重要

case	unsup. pre-train				ImageNet acc.	VOC detection		
	MLP	aug+	cos	epochs		AP ₅₀	AP	AP ₇₅
supervised					76.5	81.3	53.5	58.8
MoCo v1				200	60.6	81.5	55.9	62.6
(a)	✓			200	66.2	82.0	56.4	62.6
(b)		✓		200	63.4	82.2	56.8	63.2
(c)	✓	✓		200	67.3	82.5	57.2	63.9
(d)	✓	✓	✓	200	67.5	82.4	57.0	63.6
(e)	✓	✓	✓	800	71.1	82.5	57.4	64.0

Table 1. **Ablation of MoCo baselines**, evaluated by ResNet-50 for (i) ImageNet linear classification, and (ii) fine-tuning VOC object detection (mean of 5 trials). “**MLP**”: with an MLP head; “**aug+**”: with extra blur augmentation; “**cos**”: cosine learning rate schedule.

- MoCo vs. SimCLR vs. MoCo V2
- 将小批量与负样本量解耦，使 MoCo-V2 在较小批量下的表现
优于 SimCLR (256 v.s. 8192)

case	unsup. pre-train					ImageNet acc.
	MLP	aug+	cos	epochs	batch	
MoCo v1 [6]				200	256	60.6
SimCLR [2]	✓	✓	✓	200	256	61.9
SimCLR [2]	✓	✓	✓	200	8192	66.6
MoCo v2	✓	✓	✓	200	256	67.5
<i>results of longer unsupervised training follow:</i>						
SimCLR [2]	✓	✓	✓	1000	4096	69.3
MoCo v2	✓	✓	✓	800	256	71.1

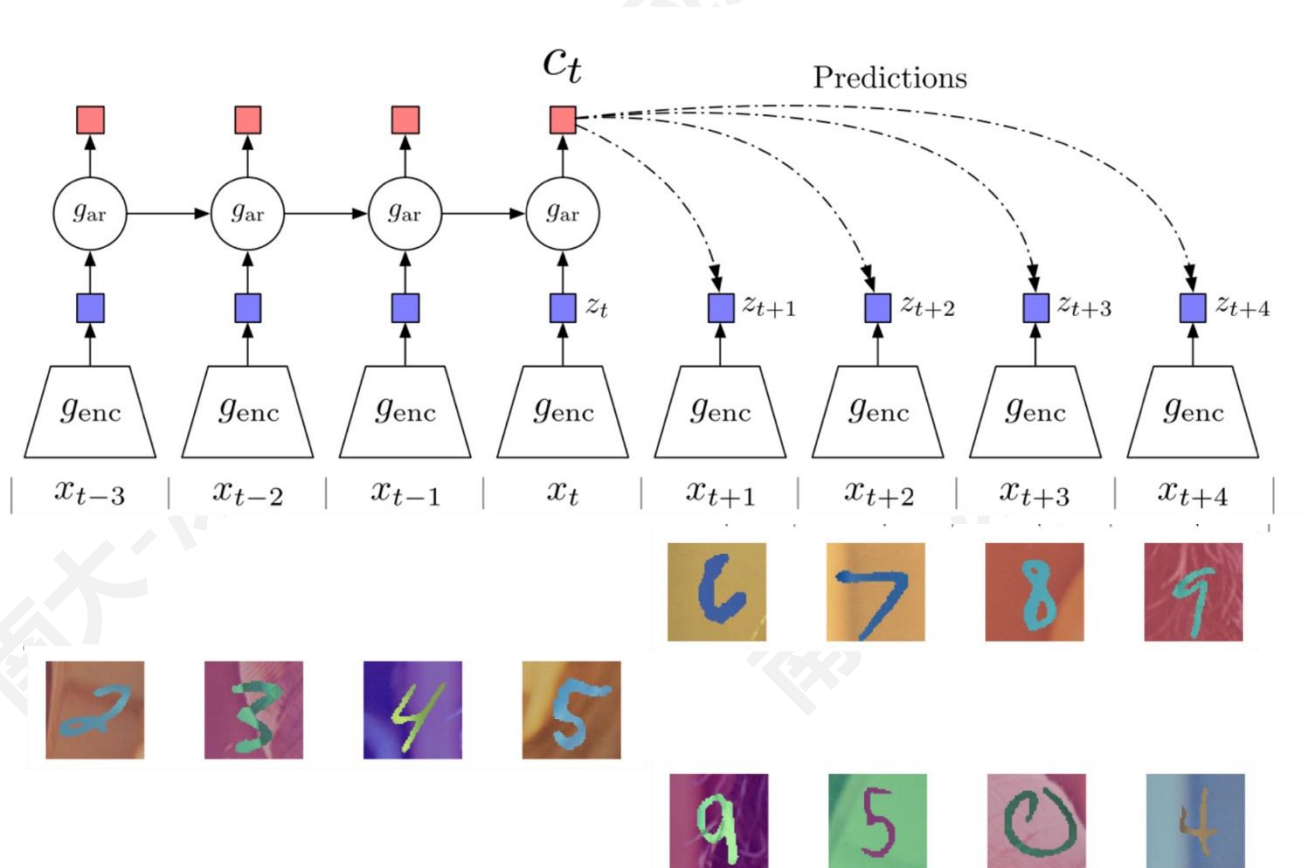
Table 2. **MoCo vs. SimCLR**: ImageNet linear classifier accuracy (**ResNet-50, 1-crop 224×224**), trained on features from unsupervised pre-training. “aug+” in SimCLR includes blur and stronger color distortion. SimCLR ablations are from Fig. 9 in [2] (we thank the authors for providing the numerical results).

- MoCo vs. SimCLR vs. MoCo V2
- 可以使用更小的显存

mechanism	batch	memory / GPU	time / 200-ep.
MoCo	256	5.0G	53 hrs
end-to-end	256	7.4G	65 hrs
end-to-end	4096	93.0G [†]	n/a

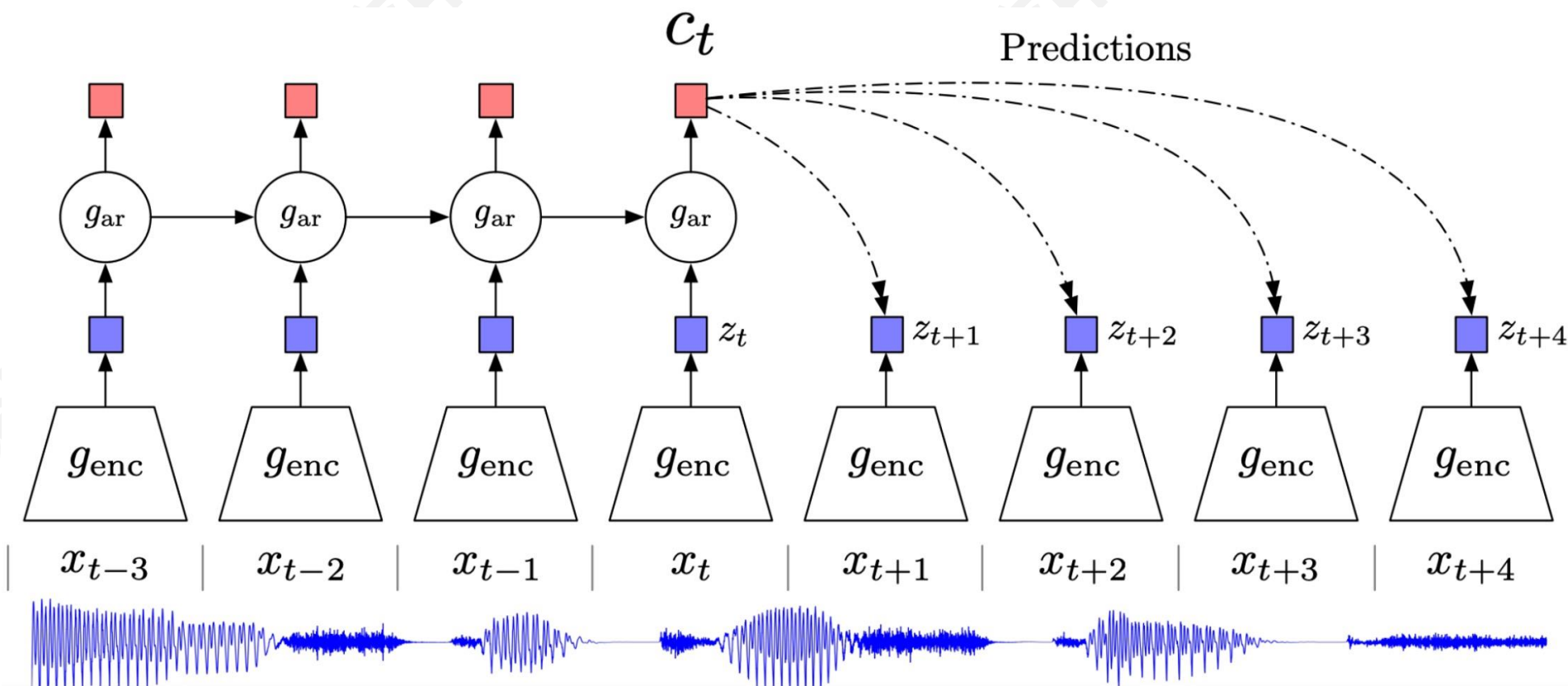
Table 3. **Memory and time cost** in 8 V100 16G GPUs, implemented in PyTorch. [†]: based on our estimation.

■ Contrastive Predictive Coding (CPC)



- 对比：使用对比学习对“正确”和“错误”序列进行对比
- 预测：模型必须在给定当前上下文的情况下预测未来的模式
- 编码：该模型为下游任务学习有用的特征向量或“代码”，类似于其他自监督方法

■ CPC 示例：音频序列建模



■ CPC 示例：音频序列建模

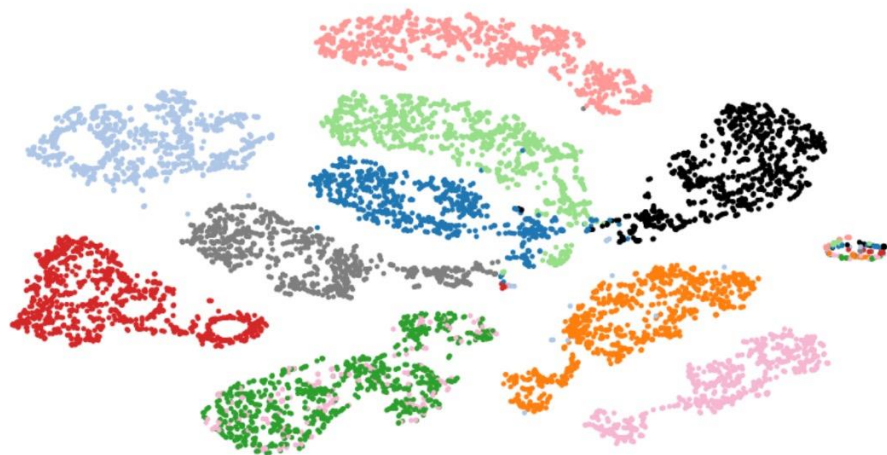
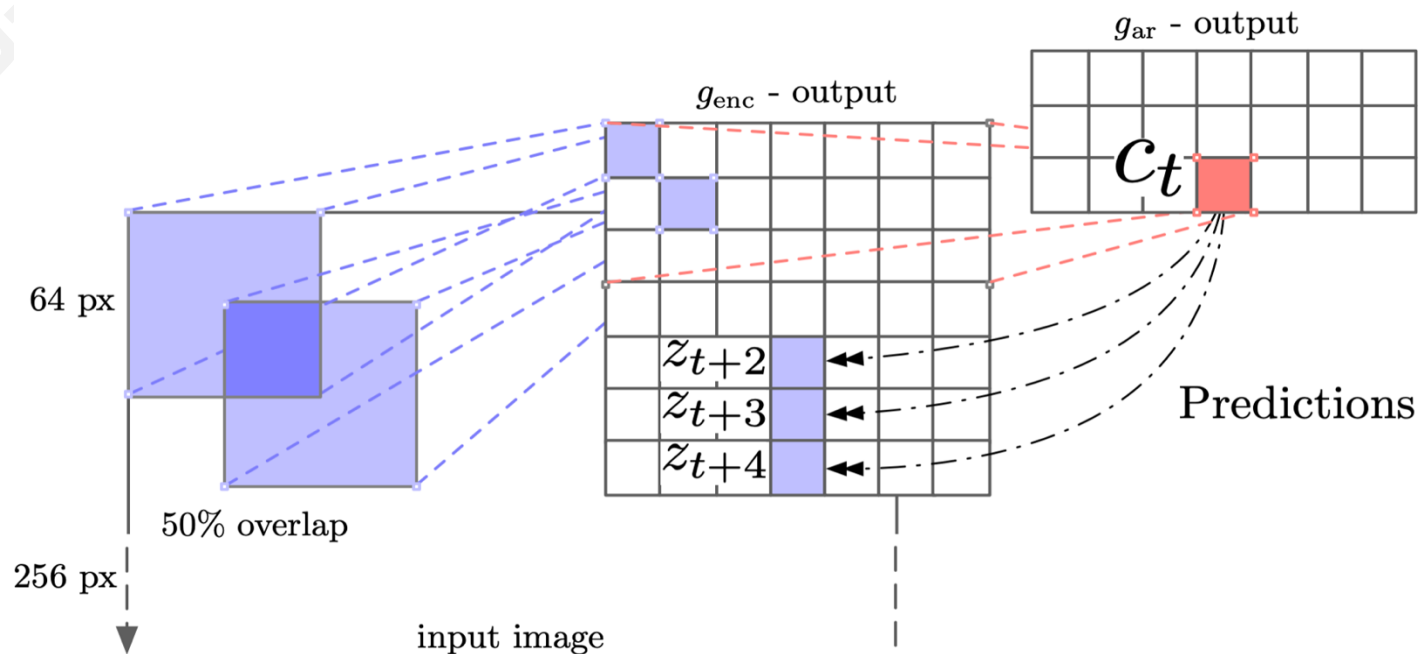


Figure 2: t-SNE visualization of audio (speech) representations for a subset of 10 speakers (out of 251). Every color represents a different speaker.

Method	ACC
Phone classification	
Random initialization	27.6
MFCC features	39.7
CPC	64.6
Supervised	74.6
Speaker classification	
Random initialization	1.87
MFCC features	17.6
CPC	97.4
Supervised	98.5

■ CPC 示例：建模视觉上下文

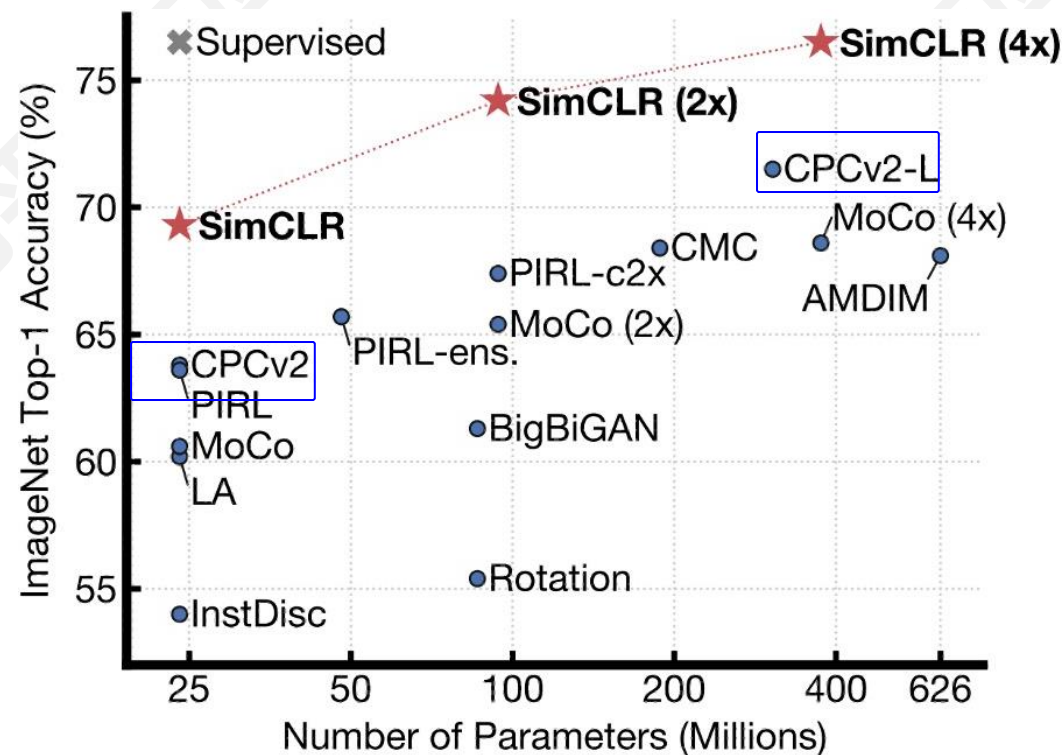
- idea: 将图像分割成图像块，将图像块行从上到下建模为一个序列。即，使用上面的行作为上下文来预测下面的行



■ CPC 示例：建模视觉上下文

Method	Top-1 ACC
Using AlexNet conv5	
Video [28]	29.8
Relative Position [11]	30.4
BiGan [35]	34.8
Colorization [10]	35.2
Jigsaw [29] *	38.1
Using ResNet-V2	
Motion Segmentation [36]	27.6
Exemplar [36]	31.5
Relative Position [36]	36.2
Colorization [36]	39.6
CPC	48.7

Table 3: ImageNet top-1 unsupervised classification results. *Jigsaw is not directly comparable to the other AlexNet results because of architectural differences.



■ MoCo V3

An Empirical Study of Training Self-Supervised Vision Transformers

Xinlei Chen* Saining Xie* Kaiming He
Facebook AI Research (FAIR)

Code: <https://github.com/facebookresearch/moco-v3>

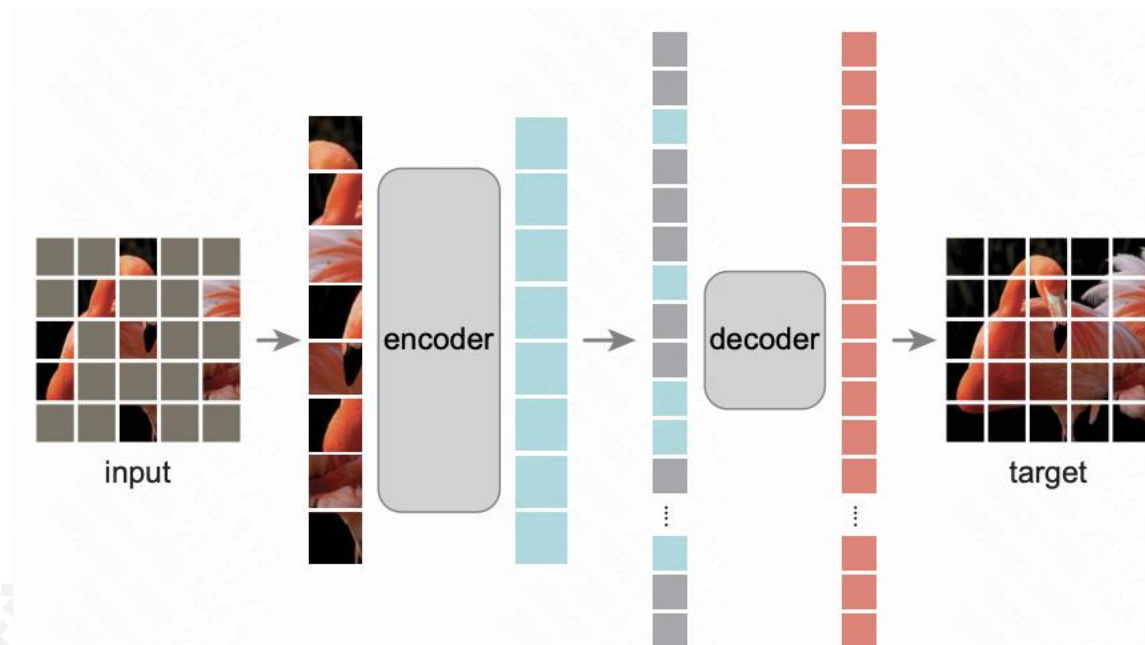
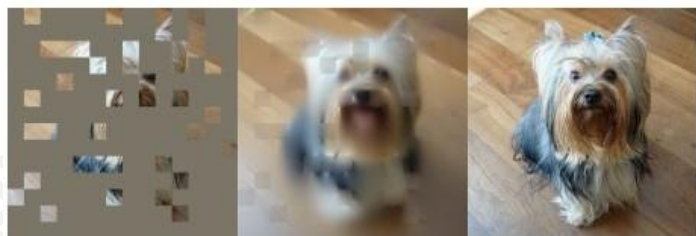
Abstract

This paper does not describe a novel method. Instead, it studies a straightforward, incremental, yet must-know baseline given the recent progress in computer vision: self-supervised learning for Vision Transformers (ViT). While the training recipes for standard convolutional networks have been highly mature and robust, the recipes for ViT are yet to be built, especially in the self-supervised scenarios where training becomes more challenging. In this work, we go back to basics and investigate the effects of several fundamental components for training self-supervised ViT. We observe that instability is a major issue that degrades accuracy, and it can be hidden by apparently good results. We reveal that these results are indeed partial failure, and they can be improved when training is made more stable. We benchmark ViT results in MoCo v3 and several other self-supervised frameworks, with ablations in various aspects. We discuss the currently positive evidence as well as challenges and open questions. We hope that this work will provide useful data points and experience for future research.

framework	model	params	acc. (%)
<i>linear probing:</i>			
iGPT [9]	iGPT-L	1362M	69.0
iGPT [9]	iGPT-XL	6801M	72.0
MoCo v3	ViT-B	86M	76.7
MoCo v3	ViT-L	304M	77.6
MoCo v3	ViT-H	632M	78.1
MoCo v3	ViT-BN-H	632M	79.1
MoCo v3	ViT-BN-L/7	304M	81.0
<i>end-to-end fine-tuning:</i>			
masked patch pred. [16]	ViT-B	86M	79.9 [†]
MoCo v3	ViT-B	86M	83.2
MoCo v3	ViT-L	304M	84.1

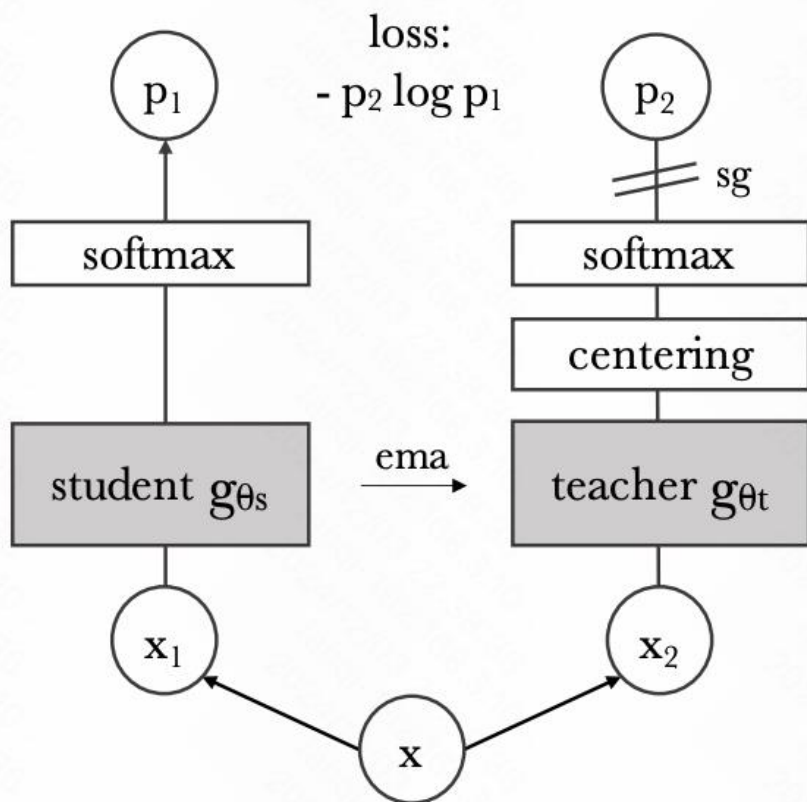
Table 1. **State-of-the-art Self-supervised Transformers** in ImageNet classification, evaluated by linear probing (top panel) or end-to-end fine-tuning (bottom panel). Both iGPT [9] and masked patch prediction [16] belong to the masked auto-encoding paradigm. MoCo v3 is a contrastive learning method that compares two (224×224) crops. ViT-B, -L, -H are the Vision Transformers proposed in [16]. ViT-BN is modified with BatchNorm, and “/7” denotes a patch size of 7×7 . [†]: pre-trained in JFT-300M.

■ Masked Autoencoder



method	pre-train data	ViT-B	ViT-L	ViT-H	ViT-H ₄₄₈
scratch, our impl.	-	82.3	82.6	83.1	-
DINO [5]	IN1K	82.8	-	-	-
MoCo v3 [9]	IN1K	83.2	84.1	-	-
BEiT [2]	IN1K+DALLE	83.2	85.2	-	-
MAE	IN1K	<u>83.6</u>	<u>85.9</u>	<u>86.9</u>	87.8

■ DINO



Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

■ DINO V2

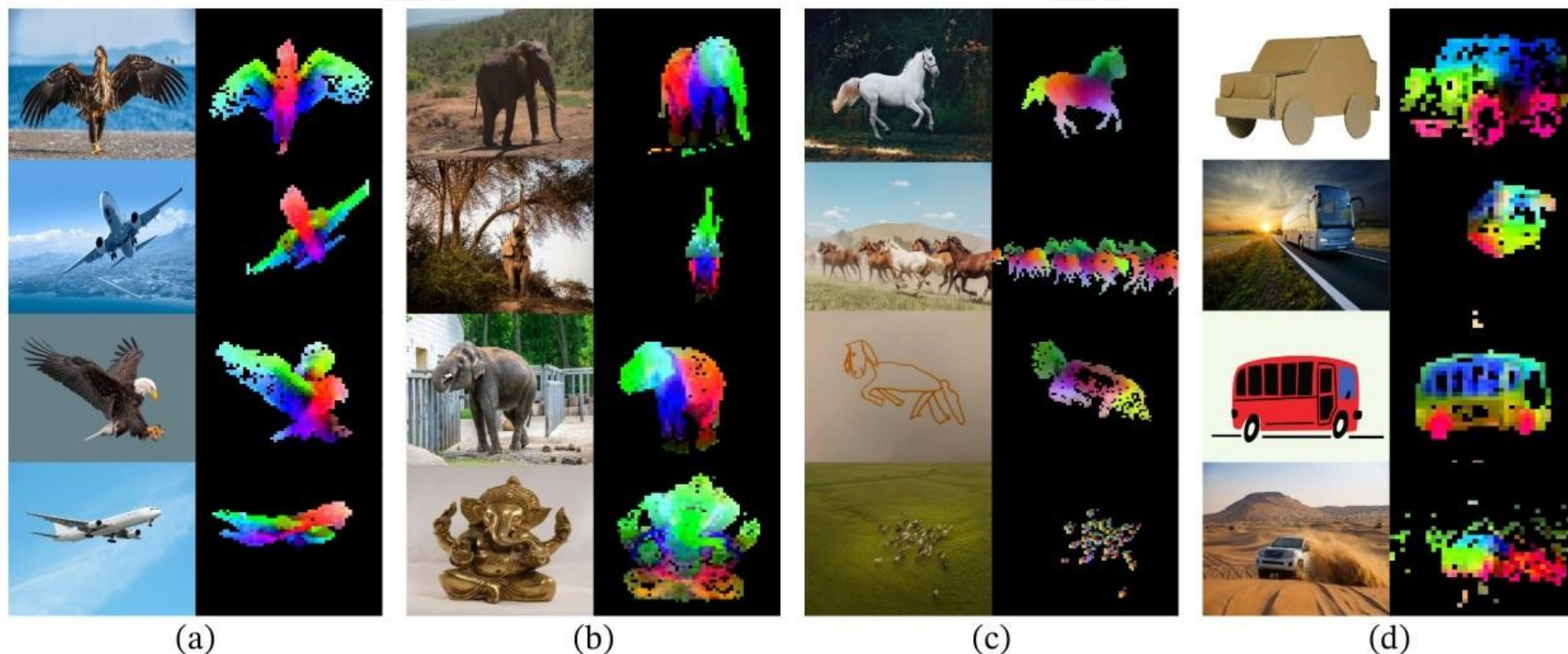
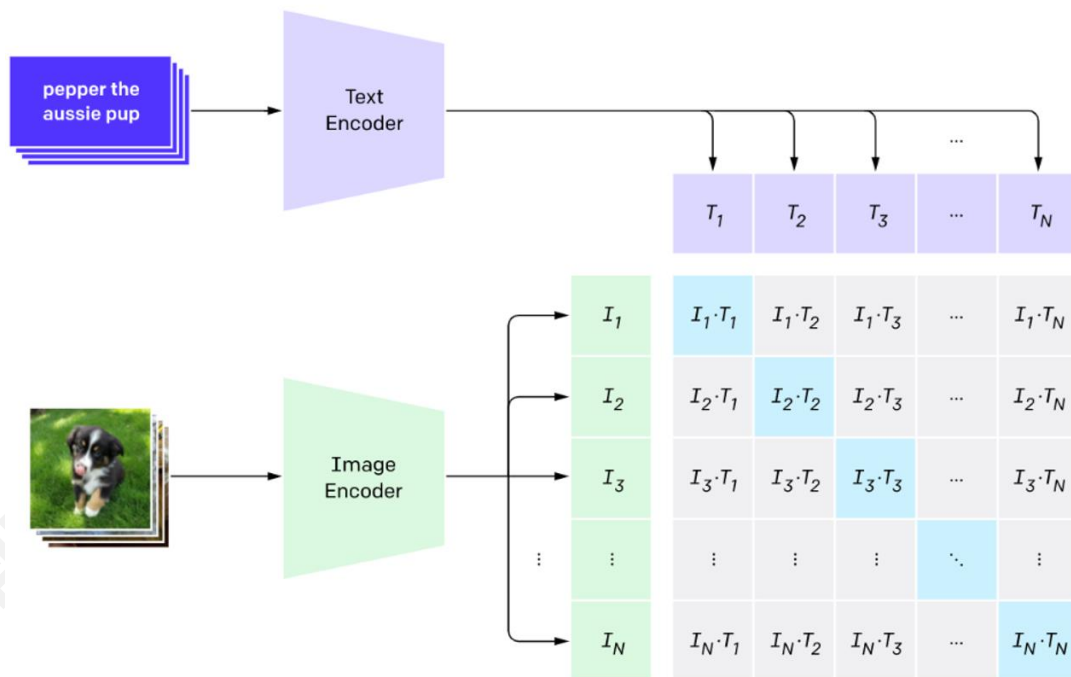


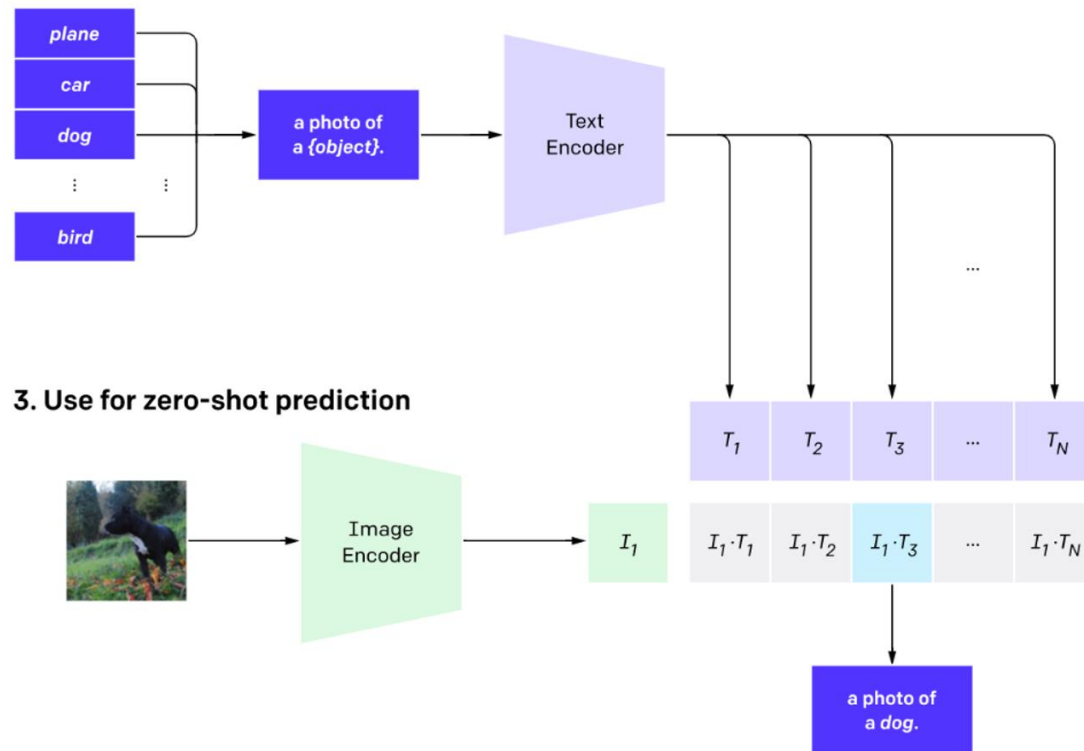
Figure 1: **Visualization of the first PCA components.** We compute a PCA between the patches of the images from the same column (a, b, c and d) and show their first 3 components. Each component is matched to a different color channel. Same parts are matched between related images despite changes of pose, style or even objects. Background is removed by thresholding the first PCA component.

CLIP

1. Contrastive pre-training



2. Create dataset classifier from label text



3. Use for zero-shot prediction